

Содержание

0.	Предисловие	4
1.	Общие сведения о языке Python	4
1.1.	Почему именно Python.....	4
1.1.1.	Преимущества Python	5
1.2.	Компилируемые и интерпретируемые языки.....	6
1.3.	Установка Python.....	7
1.4.	Первые шаги. Вывод на экран, арифметические операции и имена переменных	8
1.5.	Целочисленное деление и взятие остатка	11
1.6.	Ввод данных с клавиатуры	11
2.	Управляющие последовательности.....	15
3.	Комментарии	16
4.	Логические выражения.....	17
4.1.	Операторы И, ИЛИ и НЕ	18
4.2.	Цепочка операторов сравнения.....	19
5.	Операторы ветвления if/else.....	20
5.1.	Связанное условие elif	21
6.	Циклы while	24
6.1.	Программа для счёта факториала	26
6.2.	Задача «Герою нужно загнать валун на гору!»	27
6.3.	Комбинированный оператор присваивания	28
7.	Строки	30
7.1.	Операции над строками	30
7.1.1.	Индексирование	30
7.1.2.	Взятие длины строки len().....	30
7.1.3.	Обход строки с помощью цикла while.....	31
7.1.4.	Сложение строк (конкатенация).....	31
7.1.5.	Дублирование строк.....	32
7.1.6.	Извлечение среза	35
7.1.7.	Функции ord и chr.....	38
7.1.8.	Оператор in	39

7.2.	Строка – неизменяемый тип данных	40
7.3.	Сырые строки.....	41
7.4.	Методы строк.....	41
7.4.1.	Поиск элемента в строке – find и index.....	41
7.4.2.	Замена символов на другой - replace.....	43
7.4.3.	Разделение строки на подстроки - split.....	44
7.4.4.	Подсчет количества элементов – count.....	46
7.4.5.	Некоторые другие упрощающие работу методы.....	47
8.	Списки	50
8.1.	Вложенные списки	52
8.2.	Список – изменяемый тип данных	55
8.3.	Методы списков.....	56
8.3.1.	Добавление элемента в список – метод append	56
8.3.2.	Расширение списка – метод extend	58
8.3.3.	Удаление элемента из списка по индексу – метод pop	59
8.3.4.	Поиск элемента в списке – метод index	62
8.3.5.	Поверхностная копия списка – метод copy	63
8.3.6.	Подсчёт количества элементов – метод count.....	64
8.4.	Срезы списков.....	68
8.5.	Связь между списками и строками.....	70
8.5.1.	Преобразование строки в список – list и split.....	70
8.5.2.	Преобразования списка в строку – join.....	70
8.6.	Алгоритмы сортировки списков	72
8.6.1.	Сортировка пузырьком – Bubble sort	72
8.6.2.	Сортировка вставками – Insertion sort.....	72
8.6.3.	Быстрая сортировка – Quick sort	72
8.7.	Операции над списками.....	72
8.7.1.	Поиск максимума и минимума – max и min.....	73
8.7.2.	Сумма всех элементов – sum	73
8.7.3.	Сортировка списка – sort.....	73
8.7.4.	Объединение списков	73
9.	Цикл for	73

9.1.	Изменение параметра цикла с помощью range	76
9.2.	Перебор элементов с их индексом - enumerate.....	78
9.3.	Перебор двух списков в одном цикле - zip	80
10.	Функции.....	81
10.1.	Обязательные и необязательные параметры.	87
10.2.	Функции, как независимые сущности.....	89
10.3.	Рекурсивные функции.....	89
11.	Проект: Шифрование и дешифровка сообщений.....	90
11.1.	Маршрутный шифр	90
11.2.	Шифр жердевой изгороди	92
11.3.	Шифрование перестановочной матрицей.....	93
11.4.	Шифр redefence.....	95
11.5.	Варианты для выполнения самостоятельной работы.....	96
11.5.1.	Варианты средней сложности.....	96
11.5.2.	Варианты повышенной сложности	101
12.	Работа с файлами	103
12.1.	Текстовые файлы.....	103
12.2.	Ключевое слово with	106
12.3.	Работа с модулем os	107
13.	Словарь	107
13.1.	Получение значений из словаря.....	108
13.2.	Добавление пар в словарь.....	108
13.3.	Удаление ключа из словаря.....	109
13.4.	Работа с циклом for	109
14.	Множества	110
15.	Многофайловый проект	110
16.	Проект: Создание текстовой игры	112

0. Предисловие

Задача данной дисциплины – освоить синтаксис и инструменты, используемые в программировании на примере такого языка программирования (ЯП), как Python. В ходе первого семестра (первой части этого пособия) будут изучены основные ключевые слова и функции, различные типы данных и способы работы с ними.

1. Общие сведения о языке Python

Новые языки программирования создаются по разным причинам. В каждом из них есть своя изюминка, характерная особенность, выделяющая его среди других языков. Более того, развитие компьютерных технологий постоянно стимулирует создание новых и совершенствование имеющихся языков программирования. Разработка любого приложения подразумевает эффективное решение поставленных задач. Программистам, владеющим многими языками, проще подобрать оптимальный язык для конкретной задачи. Одни языки лучше приспособлены для работы с базами данных, другие упрощают создание элементов пользовательского интерфейса.

1.1. Почему именно Python

При создании всех языков программирования преследовались вполне определенные цели. Именно эти цели облегчают определение характеристик и способов использования языка. Ведь невозможно создать универсальный язык программирования, который позволял бы решать любые задачи. Что касается Python, то главная цель его разработки заключалась в создании языка, повышающего эффективность работы программистов. С учетом сказанного, рассмотрим основные причины выбора Python для создания приложений:

- Ускорение разработки приложений.

Код на языке Python обычно в 2-10 раз короче по сравнению с аналогичным кодом, написанным на C/C++ или Java, что позволяет сократить время разработки приложения.

- Простота чтения.

Язык программирования подобен обычному языку: нужно уметь читать тексты, написанные на нем. Код на языке Python проще читать, чем код, написанный на других языках. Это позволяет сократить время на интерпретацию кода и уделить больше времени внесению существенных изменений.

- Быстрота изучения.

Разработчики Python ставили своей целью создание простого языка, не перегруженного правилами, чтобы не усложнять его изучение. Ведь, в конце концов, задача программистов - писать приложения, а не изучать сложные языки программирования.

На август 2023 года, согласно данным организации ТЮВЕ, которая отслеживает статистику использования языков программирования, Python занимает первое место:

<https://www.tiobe.com/tiobe-index/>

1.1.1. Преимущества Python

По большому счету, создавать приложения можно на любом языке программирования. Даже если выбрать не самый удачный язык, вы все равно сможете решить поставленную задачу, просто работа затянется надолго и будет сопряжена с большим количеством ошибок. Разумеется, никому этого не хочется, поэтому важно знать, какие именно приложения чаще всего создаются с помощью Python. Ниже приведены наиболее популярные варианты.

- Создание прототипов приложений.

Разработчики часто создают прототипы, т.е. черновые варианты приложений, прежде чем приступить к написанию реальных программ. Python позволяет быстро создавать рабочие прототипы.

- Создание сценариев для веб-приложений.

JavaScript по праву считается самым популярным языком написания веб-приложений, однако Python не уступает ему в популярности, занимая почетное второе место. Python обеспечивает доступ к большому набору функций, чем JavaScript, и позволяет ускорить разработку веб-приложений (что в наши дни является большим преимуществом).

- Создание математических, научных и инженерных приложений.

Python предоставляет доступ к специализированным библиотекам, которые упрощают создание указанных приложений. Две наиболее популярные библиотеки - это NumPy (www.numpy.org) и SciPy (www.scipy.org). Благодаря им можно значительно сократить время на создание соответствующего кода.

- Работа с XML.

Расширяемый язык разметки (XML) широко применяется для хранения данных в Интернете и во многих настольных приложениях. В отличие от многих других языков программирования, в Python это базовый компонент. Таким образом, Python станет оптимальным вариантом при работе с веб-службами.

- Взаимодействие с базами данных.

Все бизнес-процессы основаны на использовании баз данных. Python не является языком запросов, как SQL или LINQ, но позволяет легко взаимодействовать с базами данных, упрощая подключения к ним.

- Разработка пользовательских интерфейсов.

В отличие от C#, в Python нет встроенного дизайнера, позволяющего перетаскивать элементы из панели инструментов. Тем не менее он располагает обширным набором графических фреймворков, представляющих собой расширения, которые упрощают создание графических интерфейсов. Некоторые из фреймворков поставляются с дизайнерами, облегчающими процесс создания пользовательских интерфейсов. Важно то, что Python не привязан к какому-то одному методу создания пользовательских интерфейсов. Вы вольны выбрать любой вариант, какой вам удобен.

1.2. Компилируемые и интерпретируемые языки

Такие языки программирования как C/C++, Java, Go, Rust являются **компилируемыми** – это означает, что исходный код программы преобразуется компилятором в машинный код, сгенерированный исполняемый файл может быть запущен операционной системой.

Python же является **интерпретируемым** ЯП – это означает, что исходный код программы выполняется напрямую интерпретатором.

Интерпретатор (interpreter) Python — это программа, которая анализирует, обрабатывает и выполняет код Python. Код выполняется построчно, это означает, что ошибка в коде зачастую будет замечена только тогда, когда исполнение кода попадёт на эту строчку.

Для справки:

Всем программистам, которые пишут на языке Python, следует придерживаться стандарта PEP-8, описанного в документе <https://www.python.org/dev/peps/pep-0008/>.

1.3. Установка Python

Для разработки программ на Python требуется установить интерпретатор Python с официального сайта:

<https://www.python.org/>

Для разработки больших и серьезных проектов недостаточно установки одного интерпретатора - разработку программ упростит удобная среда интегрированной разработки (Integrated Development Environment, IDE), которая содержит текстовый редактор с подсказками по различным возможностям и договоренностям в Python, виртуальную среду, инструментарий для системы контроля версий и др..

Такой IDE может быть PyCharm от компании JetBrains:

<https://www.jetbrains.com/ru-ru/pycharm/>

ВАЖНО:

Перед установкой PyCharm у вас должен быть установлен интерпретатор Python.

1.4. Первые шаги.

Вывод на экран, арифметические операции и имена переменных

Напишем первую программу – выведем на экран стандартный «Привет, мир!». Для этого в функцию вывода на экран `print` передадим аргумент-строку «Hello, world!»:

```
print('Hello, world!')
```

Арифметические операции поддерживаются в Python, например:

```
print(2+2)
```

В программировании всё строится на хранении где-то промежуточных данных – расчётов, текстов и др. Для хранения промежуточных данных используются переменные.

Переменная – это имя, которое ссылается на значение, хранящееся в промежуточной памяти компьютера.

Переменные можно также передавать в функции, как и сами значения строк/вычислений:

```
text = 'Результат сложения будет:'  
result = 2+3  
print(text, result)
```

Программа выведет на экран:

```
Результат сложения будет: 5
```

Правила именования переменных:

- Имена переменных могут иметь любую длину.
- Они могут начинаться с латинской буквы или знака подчеркивания «_».
- В именах переменных могут содержаться цифры, но не в начале.
- Могут использоваться как прописные (верхний регистр), так и строчные (нижний регистр) буквы в имени, но есть некоторые договоренности между программистами по наименованию

переменных. Мы их разберем позже, пока лучше используйте нижний регистр для написания имен.

Переменные должны иметь осмысленный характер, т.е. в вашем коде не должны встречаться переменные просто названные «q» или же «asde» и всё в таком духе. Имена переменных должны отражать суть для чего они используются.

Транслитерацию также запрещается использовать, названия должны быть на английском языке. Если вам надо назвать переменную, имеющую смысл «счетчик слов», то вы можете её назвать «word_counter», «word_ctr» (сокращенная версия counter), но никак не «schetchik_slov».

Также рекомендуется, чтобы переменные не занимали имена ключевых слов в Python:

False	class	finally	is	return
None	continue	for	lambda	try
True	def	from	nonlocal	while
and	del	global	not	with
as	elif	if	or	yield
assert	else	import	pass	
break	except	in	raise	

Ключевое слово – это слово, которое используется в процессе анализа исходного кода программы интерпретатором. Они зарезервированы для определенных операций, например, def для объявления функций/методов, if – для оператора ветвления и др..

Теперь попробуем наткнуться на возможные ошибки при написании простых программ! Ответьте на вопросы что произойдет если:

```
print('Hello world!')
```

1. Что произойдет с функцией вывода на экран, если вы пропустите одну из скобок? Обе?
2. Тот же вопрос, но если вы пропустите одну из кавычек? Обе?
3. Вы можете использовать знак минус, чтобы указать отрицательное число, например -2.
4. Что произойдет, если вы укажете знак плюс перед числом?
5. К чему приведет код 2++2?
6. В математике нули в начале — это абсолютно нормально, например, так: 02. Что произойдет, если вы попробуете это сделать в Python?
7. Что произойдет, если указать два значения без оператора между ними?

Написать программы, которые будут вычислять:

$a=100$ $c = a - 5*10$ Вывести на экран значение c	$x = 10 + 60$ $z = 100 - x$ Вывести на экран значение z	$a = 300-256$ $b = 5*15$ $c = a+b$ Вывести на экран значение c
$a = 10*5 - 3$ $b = 20/4$ $c = 100$ $d = a+(b-c)$ Вывести на экран значения a, b, c, d	$k = 100/10 + 1$ $m = k - 15$ $n = k * 2$ $p = k*m/n$ Вывести на экран значения k, m, n, p	$q = 69/3$ $w = 23*4-q$ $e = q + w * (-2)$ $r = q/w + e/w$ Вывести на экран значения q, w, e, r

Переменные могут быть различных типов в зависимости от того, что нужно хранить в них. Приведем ниже базовые типы данных:

Название типа	Обозначение	Расшифровка	Пример значений
Целое число	int	Сокращенно от integer, «целое число»	1, 26, -52, 0
Вещественное число (десятичные дроби)	float	Переводится как «плавать», вещественные числа так и называются «числа с плавающей точкой»	2.5, 1.234, -2.6, 3.0

Текстовый/ строковый	str	Сокращенно от string, «нить»	‘яблоко’, ‘кот’, ‘т’
Булевый (логическое ЛОЖЬ или ИСТИНА)	bool	Название идет от булевой алгебры, раздела математики, который изучает высказывания со стороны их логических значений (истины и лжи)	True (переводится как истина), False (переводится как ложь)

1.5. Целочисленное деление и взятие остатка

```
x = 56 / 3
```

```
print(x)
```

```
x = 56 // 3
```

```
print(x)
```

```
x = 56 % 3
```

```
print(x)
```

```
x, y = divmod(56, 3)
```

```
print(x, y)
```

1.6. Ввод данных с клавиатуры

Для того чтобы ввести данные с клавиатуры воспользуемся функцией `input`, например:

```
print('Введите число:')
x = input()
```

```
print('Введите слово:')
y = input()
print('Вы ввели', x, y)
```

Примером работы программы может быть:

```
Введите число:
5
Введите слово:
apple
Вы ввели 5 apple
```

Согласитесь, не очень удобно выводить на экран информацию для ввода с клавиатуры отдельной строчкой `print`. Их можно объединить, передав текст в функцию `input`. Попробуйте такой код:

```
x=input('Введите число: ')
y=input('Введите слово: ')
print('Вы ввели', x, y)
```

Есть небольшая разница – теперь ввод значений происходит на той же строчке, где и текст. Это может быть и удобнее для пользователя:

```
Введите число: 2
Введите слово: carrot
Вы ввели 2 carrot
```

Хорошо, раз мы можем так получить число, то попробуем использовать его в вычислениях. Считаем с клавиатуры число и добавим к нему 2.

Запустите такой код:

```
x=input('Введите число: ')
k=x+2
print(x, '+ 2 =', k)
```

Что получилось? Уверен, у вас возникла ошибка:

```
TypeError: can only concatenate str (not "int") to str
```

Поучимся их читать:

«Ошибка типа: сложение строки может происходить только со строкой, не целым числом»

Проверим какого типа значение в переменной x после ввода с клавиатуры:

```
x=input('Введите число: ')
print(type(x))
```

На экране будет выведен тип переменной:

```
<class 'str'>
```

Если вы ещё не догадались – проговорим, функция `input` всегда возвращает значения строкового типа. Для того чтобы использовать строку «2» как число 2 нужно преобразовать текст в число, для этого нужно использовать явное преобразование в число `int()`:

```
x=input('Введите число: ')
x=int(x) # Преобразуем в число!
k=x+2
print(x, '+ 2 =', k)
```

Программа успешно выполнит такой код:

```
Введите число: 2
2 + 2 = 4
```

Но этот код можно записать более компактно – в одной строчке сначала вызвать ввод с клавиатуры, а затем преобразование к числу:

```
x=int(input('Введите число: '))
k=x+2
print(x, '+ 2 =', k)
```

Попрактикуемся:

Задание 1. Переменная `k` задаётся с клавиатуры. Программа должна вычислять $b = k*3+5-20$. Вывести на экран `b`.

Задание 2. Объем сферы с радиусом r составляет $\frac{4}{3} * \pi * r^3$. Каков объем сферы с радиусом 5?

Для возведения в степень вы можете воспользоваться двумя разными способами:

- первый – использовать два идущих подряд знака умножения, т.е. «r**3»;
- второй – использовать функцию pow, т.е. «pow(r, 3)».

Задание 3. Предположим, что книга стоит 249 рублей 50 копеек, при этом книжный магазин предоставляет скидку в 40%. Стоимость доставки составляет 100 рублей за первый экземпляр и 49 рублей 50 копеек за каждый дополнительный. В какую сумму обойдется закупка 60 экземпляров?

Задание 4. Если я вышел из дома в 6:52 утра и пробежал 1 км в легком темпе (1 км за 8 мин. 15 сек.), потом 3 км в среднем темпе (1 км за 7 мин. 12 сек.) и 1 км в легком темпе снова, то во сколько я закончу пробежку?

2. Управляющие последовательности

Некоторые символы невозможно отобразить обычными методами, например, для того, чтобы обозначить строку, рекомендую использовать одинарную кавычку «'», хотя в Python можно использовать и двойную «"», для интерпретатора не важно какую из них вы будете использовать и если вы захотите при рекомендуемой записи строки вывести сообщение «Название романа Льва Толстого "Война и мир" скрывает глубокий философский смысл» нужно написать код:

```
print('Название романа Льва Толстого "Война и мир" скрывает  
глубокий философский смысл')
```

Но что если вы захотите использовать одинарные кавычки для выделения наименования романа?

```
print('Название романа Льва Толстого 'Война и мир' скрывает  
глубокий философский смысл')
```

Такой код работать не будет, интерпретатор выделит строки так:

```
print('Название романа Льва Толстого 'Война и мир' скрывает  
глубокий философский смысл')
```

Т.е. получится, что в коде теперь присутствуют две строки и неясный текст «Война и мир» между ними. Для того, чтобы избавиться от такого непонимания нужно объяснить интерпретатору, что данные одинарные кавычки нужно воспринимать, как обычные символы, которые не имеют какого-либо специального смысла, нужно поставить перед кавычкой символ слеша, чтобы получилось «\»:

```
print('Название романа Льва Толстого \'Война и мир\' скрывает  
глубокий философский смысл')
```

Специальные символы, которые вызываются с помощью слеша, называются **управляющими последовательностями**.

Управляющая последовательность	Значение символа
\'	Одинарная кавычка
\"	Двойная кавычка
\n	Перевод каретки на новую строку
\t	Табуляция
\\	Обратная косая черта

Попрактикуемся:

Задание 1. Выведите на экран слова «апельсины», «мандарины», «сливы», разделенные друг от друга символом табуляции.

Задание 2. Выведите на экран слова «огурцы», «арбузы», «ежевика» на разных строках с помощью символа перевода каретки.

Задание 3. Выведите на экран путь к файлу: «C:\Users\tables\nuggets\file.txt». Разберитесь самостоятельно какая управляющая последовательность должна быть использована, чтобы вывод пути был корректным.

3. Комментарии

Закомментировать строку - #

```
x = 5#+2  
print(x)
```

Три кавычки подряд – многострочный комментарий

''' – Открывает комментарий

''' – Закрывает комментарий

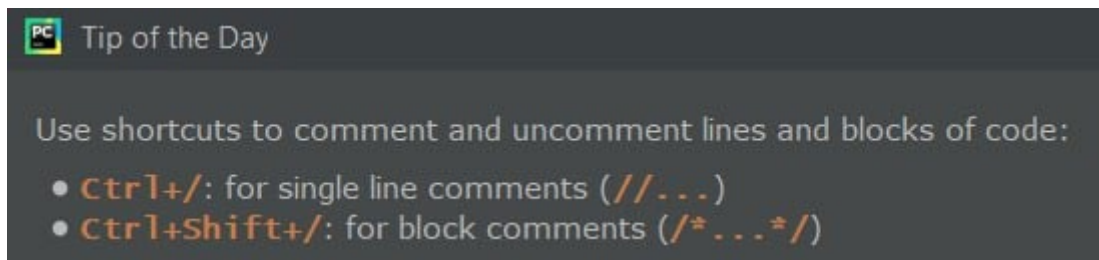
```
x = 5+2
```



```
'''  
x=22  
y=100*20  
...  
'''  
  
y=22  
print(x,y)
```

Работа в PyCharm:

Ctrl + /



4. Логические выражения

Логическим (или булевым) называется выражение, которое может быть истинным или ложным.

Запишем операторы сравнения:

- $x == y$ # x равно y
- $x != y$ # x не равно y
- $x > y$ # x больше, чем y
- $x < y$ # x меньше, чем y
- $x >= y$ # x больше или равно y
- $x <= y$ # x меньше или равно y

```
answer = 5 <= 6  
print(answer)
```

Проверим, сравните на Пайтон числа. Результаты логических выражений записывайте также в переменную `answer`, как в примере выше:

- 5 больше 3
- 11 меньше -1
- 600 больше или равно 600.23
- 53 равно 53
- 11 не равно 0

4.1. Операторы И, ИЛИ и НЕ

Существует три логических оператора: `and` (И), `or` (ИЛИ) и `not` (НЕ). С помощью И и ИЛИ можно объединять два логических выражения в одно:

```
answer = (5 <= 6) and (11>20)
print(answer)
```

Оператор НЕ используется для обращения выражения в обратное значение (ложь обращает в истину и наоборот).

```
answer = not (11>20)
print(answer)
```

Примените операторы на практике, составьте программы, которые выводят результаты:

1. 5 меньше 6 И 11 больше -1
2. 50 больше или равно 51 ИЛИ НЕ 11 равно 20
3. 13 больше 16 ИЛИ 21 равно 22
4. 'апельсин' равняется 'АпЕЛЬсиН'.lower() И ord('a') равняется 97

Таблица истинности AND (И), OR (ИЛИ) и NOT (НЕ)

Составим таблицу истинности для вышеуказанных операторов. Обозначим 0 за False или же «Ложь», а 1 за True или же «Истина».

A	B	A AND B	A OR B	NOT(A)
0	0	0	0	1
0	1	0	1	1
1	0	0	1	0
1	1	1	1	0

Проговорим что происходит в таблице истинности: оператор И является можно представить, как двоичное умножение, поэтому истинным будет только то выражение, в котором оба элемента истинны. Оператор ИЛИ является двоичной суммой, а потому результат будет всегда истинный, если хотя бы один элемент – истина. Оператор НЕ просто же обращает ложь в истину, а истину в ложь.

4.2. Цепочка операторов сравнения

Напишем программу, которая проверяет находится ли введенное число в диапазоне от 3 до 10 с помощью оператора И.

```
x = int(input('Введите число в диапазоне от 3 до 10: '))
range_check = x>=3 and x<10
print(f'Ответ будет: {range_check}')
```

Такую программу можно написать изящнее с использованием цепочки операторов сравнения. Т.е. можно сразу сделать несколько сравнений, если они связаны по смыслу, сравнение « $x \geq 3$ И $x < 10$ » записывается как « $3 \leq x < 10$ »:

```
x = int(input('Введите число в диапазоне от 3 до 10: '))
```

```
range_check = 3 <= x < 10
print(f'Ответ будет: {range_check}')
```

Цепочка может быть куда больше. Например, нужно проверить, что x меньше y , y меньше z , z меньше d , d меньше a , т.е. был бы код:

```
x = -3
y = 1
z = 10
d = 12.5
a = -1
check = x < y < z < d < a
print(f'Итоговый ответ: {check}')
```

Причем, синтаксис цепочки сравнений таков, например, для переменных x, y, z :

x <оператор сравнения> y <оператор сравнения> z

Т.е. между ними может стоять любой оператор сравнения, а значит возможны такие строчки (и не только они):

```
x < y < z
x > y > z
x == y != z
x >= y <= z
x < y > z
```

5. Операторы ветвления if/else

Для того чтобы изменять поведение программы в зависимости от различных условий можно воспользоваться операторами ветвления if/else:

- оператор if означает «если», эта ветка будет выполняться если логическое выражение является истинным;
- оператор else - альтернативное выполнение, означает «иначе», эта ветка будет выполняться если логическое выражение в if оказалось ложью.

Структуру оператора ветвления можно описать на формальном языке так:

```
if <логическое выражение>:
    <Выполняемые действия если условие истинно>
else:
    <Выполняемые действия если условие ложно>
```

В качестве первого примера напишем программу, которая получает на вход от пользователя некоторое число и затем проверяет больше ли оно 6. Если это так, то вывести «<введенное число> больше 6», иначе вывести «<введенное число> меньше 6».

```
x = int(input('Число для сравнения: '))
if x>6:
    print(x, 'больше 6')
else:
    print(x, 'меньше 6')
```

Пример работы программы:

Вариант 1	Вариант 2
Число для сравнения: 9 9 больше 6	Число для сравнения: 1 1 меньше 6

Казалось бы, задача решена и все отлично! Но как бы не так, программисты должны продумывать также и неуспешные сценарии программы – т.е. находить места, в которых программа может повести себя некорректно. Например, что будет, если пользователь введет число 6?

Результатом работы программы будет:

```
Число для сравнения: 6
6 меньше 6
```

Конечно, это неверный ответ. Мы нашли один из «багов» программы. Для того чтобы решить эту проблему добавим ещё одно условие:

```
x = int(input('Число для сравнения: '))
if x>6:
    print(x, 'больше 6')
else:
    if x==6:
        print(x, 'равно 6')
    else:
        print(x, 'меньше 6')
```

5.1. Связанное условие elif

Такое решение является не особо изящным - воспользуемся структурой if/elif/else. Благодаря elif, связанного условия, мы можем добавлять

дополнительные условия, если условие if не сработало. Код в else выполнится только если любое другое условие if/elif не подошло. На формальном языке это можно описать так:

```
if <логическое выражение 1>:
    <Код выполняется если ЛВ 1 истинно>
elif <логическое выражение 2>:
    <Код выполняется если ЛВ 2 истинно>
elif <логическое выражение 3>:
    <Код выполняется если ЛВ 3 истинно>
else:
    <Код выполняется если ни одно из ЛВ выше не было истинно>
```

В такой структуре будет выполнен код только у первого подходящего условия. Т.е. если ЛВ 2 и ЛВ 3 будет истинно, то выполнится только код ЛВ 2, т.к. оно выше. Доработаем код с применением if/elif/else:

```
x = int(input('Число для сравнения: '))
if x>6:
    print(x, 'больше 6')
elif x==6:
    print(x, 'равно 6')
else:
    print(x, 'меньше 6')
```

Также переменные в строку можно вставлять более удобным способом – с помощью f-строк. Пример f-строки:

```
x=23
print(f'внутри текста {x} влезло число')
```

Т.е. чтобы вставить значение переменной в строку нужно поставить перед ней букву f (обозначив интерпретатору что далее будет f-строка), а в самом тексте вставить переменную, заключенную в фигурные скобки.

```
x = int(input('Число для сравнения: '))
if x>6:
    print(f'{x} больше 6')
elif x==6:
    print(f'{x} равно 6')
else:
    print(f'{x} меньше 6')
```

Напишем программу, проверяющую верность вычислений пользователем:

```
x=int(input('11+3-10='))
if x == (11+3-10):
    print('Вы верно решили пример!')
else:
```

```
print('Пример решён неверно')
```

Можем усложнить этот пример, для этого используем библиотеку для получения случайных значений – random. Для начала напишем код, который генерирует случайное значение от 0 по 100:

```
import random
x = random.randint(0,100)
output_text = f'Ответ будет: {x}'
print(output_text)
```

Для того чтобы воспользоваться библиотекой random нам нужно было её подключить, поэтому мы использовали директиву import, что и переводится как «подключи/добавь».

Библиотека содержит функции, переменные и константы, определенные в модуле. Чтобы получить доступ к одной из функций, вы должны указать имя библиотеки и имя функции, разделенные точкой. Такой формат называется *точечной нотацией* (dot notation).

Попрактикуемся:

Задача 1. Теперь попробуйте объединить два предыдущих кода в один. Сделаем программу, которая позволяет тренировать устный счет пользователя. Программа должна генерировать три случайных значения x,y,z, таким образом при каждом запуске программы будет создаваться новый случайный пример $x+y-z$. Результат решения примера будет сравниваться с вводом пользователя. Вывести пользователю сообщение о верном или неверном решении примера. В любом случае вывести что пользователь ввел и какой ответ был верный.

Примеры работы программы:

Неверное решение	Верное решение
Решите пример: $79+68-95$ 22 Неверно! Вы ввели: 22 Ответ был: 52	Решите пример: $11+10-3$ 18 Верно! Вы ввели: 18 Ответ был: 18

Задача 2. Требуется создать программу для проверки соответствия длины досок, поставляемых на склад. Вначале на вход программы подается два значения – нижнее и верхнее значение диапазона допустимых значений. Затем передается три значения – длины трех досок одной поставки.

Программа должна проверить каждое из значений на допустимость заданному диапазону.

Также необходимо удостовериться, что диапазон, заданный значениями a и b , выполняет условие $a \leq b$.

Входные значения	Выходные значения
Введите диапазон: 4 20 Введите значения трех досок: 2 11 4	Доска длиной 2 недопустима Доска длиной 11 допустима Доска длиной 4 допустима
Введите диапазон: 1 5 Введите значения трех досок: 2 4 11	Доска длиной 2 допустима Доска длиной 4 допустима Доска длиной 11 недопустима
Введите диапазон: 20 1 Введите значения трех досок: 100 0 45	Неверное значение диапазона

6. Циклы while

Цикл while (с английского «пока») будет выполняться до того момента, *пока* выполняется записанное в `while` условие.

Переменная цикла – это такая переменная, от состояния которой зависит условие завершения цикла.

Итерация - выполнение набора инструкций с использованием цикла.

Бесконечный цикл - цикл, в котором условие завершения никогда не выполняется.

Рассмотрим восходящий цикл. **Восходящий цикл** – это такой цикл, в котором значение параметра цикла увеличивается с каждой итерацией.

Например, пока n меньше 10, вывести n. С каждым ходом цикла (итерацией) вывести n и увеличивать n на 1.

```
n=0
while n<10:
    print(n)
    n = n + 1
```

При необходимости, можно циклы выполнять и в обратную сторону. Например, пользователь вводит число, мы его уменьшаем с каждым шагом. Цикл выполняется, пока число больше нуля.

```
k=int(input())
while k>0:
    print(k)
    k=k-1
```

Сейчас пример выше ничего не значит, но предположим - вам дают 12 долек яблока, вы будете есть их пока дольки не закончатся, смоделируем эту ситуацию:

```
apple_slice=12
while apple_slice>0:
    print(f'У вас сейчас {apple_slice} долек')
    print('Съел 1 дольку')
    apple_slice = apple_slice - 1
print('Все дольки съедены!')
```

Есть инструкция, позволяющая нарочно выйти из цикла (break). Такое мы использовать не будем. Если вы не можете продумать грамотно условие выхода из цикла – вы плохо продумали алгоритм выполнения программы.

6.1. Программа для счёта факториала

Факториал представляет из себя произведение последовательности чисел:

$$1! = 1$$

$$2! = 1 * 2$$

$$3! = 1 * 2 * 3$$

$$4! = 1 * 2 * 3 * 4$$

$$6! = 1 * 2 * 3 * 4 * 5 * 6$$

...

Написать программу, которая с помощью цикла `while` позволяет посчитать факториал числа, введённого с клавиатуры.

Пример работы программы:

Введите число: 6

Факториал числа: 720

6.2. Задача «Герою нужно загнать валун на гору!»



Представим ситуацию – герою нужно загнать валун на гору, и помочь ему в этом может только пользователь. У героя есть показатель прогресса катания валуна – `hero_progress`, изначально он равен 25.

Если игрок вводит 'z', то прогресс увеличивается на 25. Если же другую клавишу, то прогресс уменьшается на 12.5.

Игра не окончится пока игрок не закатит валун на гору (`hero_progress` должен быть больше или равен 100).

Сообщать игроку его прогресс на каждой итерации, а также вернуть нажал ли кнопку и на сколько изменится прогресс катания. Каждый раз, когда ожидается ввод 'z', сообщать об этом.

6.3. Комбинированный оператор присваивания

Комбинированным операторами присваивания, КОП (англ. «augmented assignment statements») - называются такие операторы, которые совмещают одновременно некоторую операцию и операцию присваивания. Например, одновременно сумму и присваивание с помощью КОП можно записать как «+=». Этот и некоторые другие КОП приведены ниже:

Запись	Значение	Пример	Результат
<code>x += y</code>	Добавить к <code>x</code> значение <code>y</code>	<code>x = 10</code> <code>x += 3</code>	13
<code>x -= y</code>	Вычесть из <code>x</code> значение <code>y</code>	<code>x = 10</code> <code>x -= 3</code>	7
<code>x *= y</code>	Умножить <code>x</code> на <code>y</code>	<code>x = 10</code> <code>x *= 3</code>	30
<code>x /= y</code>	Разделить <code>x</code> на <code>y</code>	<code>x = 10</code> <code>x /= 3</code>	3.3333333333333335
<code>x //= y</code>	Выполнить целочисленное деление <code>x</code> на <code>y</code>	<code>x = 10</code> <code>x //= 3</code>	3
<code>x %= y</code>	Получить остаток от деления и записать в <code>x</code>	<code>x = 10</code> <code>x %= 3</code>	1
<code>x &= y</code>	Бинарное И	<code>x = True</code> <code>x &= False</code>	False
<code>x = y</code>	Бинарное ИЛИ	<code>x = True</code> <code>x = False</code>	True
<code>x **= y</code>	Возведение в степень	<code>x = 10</code> <code>x **= 3</code>	1000

Есть пара операций, которые хотелось бы отметить отдельно:

Инкремент – операция, которая увеличивает значение переменной на единицу.

$$x = x + 1$$

Важно: Инкремент можно записать с помощью КОП:

$$x += 1$$

Декремент - операция, которое уменьшает значение переменной.

$$x = x - 1$$

Важно: Декремент можно записать с помощью КОП

$$x -= 1$$

7. Строки

Строка — это упорядоченная неизменяемая последовательность символьных значений.

7.1. Операции над строками

7.1.1. Индексирование

Выполним операцию индексирования (обращение к порядковому элементу строки), чтобы получить отдельный элемент строки.

```
fruit = 'банан'
letter = fruit[1]
print(letter)
```

Ответим на вопросы:

1. Что будет выведено на экран?
2. Что будет если обратиться по индексу 5?
3. Что будет если обратиться по индексу -1? -5? -10?

Б	А	Н	А	Н
0	1	2	3	4
-5	-4	-3	-2	-1

7.1.2. Взятие длины строки len()

Функция len(), применяемая к строке, выведет длину строки или же количество символов в ней.

```
a = len(fruit)
```

```
print(a)
```

Попрактикуемся:

Задание 1. Число вводится с клавиатуры, если оно меньше длины строки 'Клетка', то вывести элемент под этим индексом. Иначе – «Ошибка. Выход за границы строки».

Задание 2. В созданную программу из задания 1 попробуйте ввести «-1», так вы увидите, что программа работает и с отрицательными значениями. Доработайте программу, чтобы при вводе, например, «-10» выводилась такое же сообщение «Ошибка. Выход за границы строки».

7.1.3. Обход строки с помощью цикла while

'Кошка'

0 К

1 о

2 ш

3 к

4 а

7.1.4. Сложение строк (конкатенация)

Строки можно складывать друг с другом, второй элемент сложения просто добавится с конца первого

Например:

```
a = 'fruit'  
b = 'salad'  
print(a + b)
```

Попрактикуемся:

Задание 1. В программе выше между двумя словами должен стоять пробел. Придумайте как его туда поставить. Можно применять **только** сложение строк.

Задание 2. На вход подается строка из чисел, разделенных точкой с запятой. Требуется получить новую строку так, чтобы каждый нечетный разделитель убирался, таким образом объединяя два числа в одно, а каждый четный разделитель заменялся на тире.

Входные данные	Выходные данные
55;22	5522
112;22;44;546433	11222-44546433
9136;5372;481;7677;1765;1596;8620; 3684;5971;3580;4393	91365372-4817677- 17651596-86203684- 59713580-4393

Задание 3. Используя конкатенацию строк и цикл `while`, создайте программу, которая переводит любое десятичное число в двоичное.

7.1.5. Дублирование строк

Строки можно дублировать, умножая их на численные значения.

Например:

```
text = 'spam'
print(text*3)
```

Зачем такое нужно?

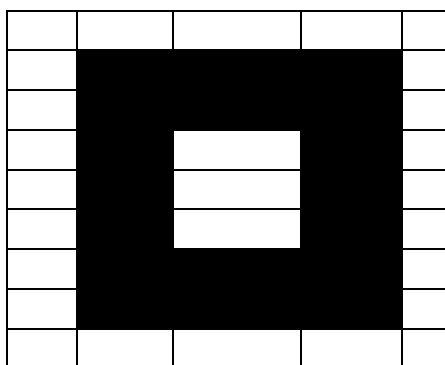
Дублирование можно применить для оформления консольных приложений, пример:

```
print('-'*25)
```



```
print('Login: Username')
print('Email: Email@gmail.com')
x=input('Введите свой пароль:\n')
print('-'*25)
print('Данные корректно введены')
```

Или же для вывода картинок в консоли – псевдографики, у которой в качестве примитивов используются различные символы. Например, нарисуем цифру ноль:



```
print('-'*20)
print('-'*4 + '*'*12 + '-'*4)
print('-'*4 + '*'*12 + '-'*4)
print('-'*4 + '*'*4 + '-'*4 + '*'*4 + '-'*4)
print('-'*4 + '*'*4 + '-'*4 + '*'*4 + '-'*4)
print('-'*4 + '*'*4 + '-'*4 + '*'*4 + '-'*4)
print('-'*4 + '*'*12 + '-'*4)
print('-'*4 + '*'*12 + '-'*4)
print('-'*20)
```

При выполнении такого кода в консоли получится:

Попрактикуйтесь:

Задача 1. Напишите код, который позволит вывести на экран любую другую цифру или символ.

Задача 2. Используя цикл `while` и дублирование строк, напишите программу, которая строит картинку треугольника из звездочек, в зависимости от введенного числа с клавиатуры.

Ввод пользователя	Результат работы программы
Задайте основание треугольника: 5	***** **** *** ** *
Задайте основание треугольника: 3	*** ** *

Задача 3. Измените программу задания 2 так, чтобы строился перевернутый треугольник:

Ввод пользователя	Результат работы программы
Задайте основание треугольника: 5	* ** *** **** *****
Задайте основание треугольника: 3	* ** ***

Задача 4. Напишите программу, которая строит шахматную доску $N \times N$, размер клеток также задаётся как $M \times M$.

Ввод пользователя	Результат работы программы
Введите размер доски: 2	***--
Введите размер клеток: 3	***--

	<pre> ***-- ---*** ---*** ---*** </pre>
Введите размер доски: 6 Введите размер клеток: 2	<pre> **--**--**-- **--**--**-- --**--**--** --**--**--** **--**--**-- **--**--**-- --**--**--** --**--**--** **--**--**-- **--**--**-- --**--**--** --**--**--** </pre>

7.1.6. Извлечение среза

Извлечение среза – это операция, позволяющая взять подстроку от основной строки.

Кратко операция записывается как: «value[START : STOP]», где:

- value – имя переменной;
- START – это начало среза;
- STOP – это конец среза.

Читается так «*взять срез от START до STOP*». Не забывайте разницу между «*по*» и «*до*». «*До*» - означает, что элемент не включается.

Опробуем срезы на практике:

```

fruit = 'banana'
print(fruit[0:3])
print(fruit[3:5])

```

Срезы можно применять для вставки внутрь строки другой строки. Например, в текст «Пробежка днем» нужно добавить «утром и», чтобы получилось «Пробежка утром и днем». Попробуйте сделать такую программу.

Самое главное следить за индексами – нужно взять срез от 0 до 8 и срез от 9 до 13.

Т.е. у вас получится примерно такой код:

```
text = 'Пробежка днем'
p1 = text[0:8]
p2 = text[9:13]
print(p1+' утром и '+p2)
```

Результат работы программы:

Пробежка утром и днем

Отметим, что если начало среза начинается с нулевого элемента, то обязательно писать ноль:

```
print(fruit[:3])
```

Аналогично с концом строки. Если срез берется до конца, то эту позицию также можно оставить пустой:

```
print(fruit[3:])
```

Тогда код из примера выше можно переписать так:

```
text = 'Пробежка днем'
p1 = text[:8]
p2 = text[9:]
print(p1+' утром и '+p2)
```

Если индекс начала среза больше или равно концу среза, то результирующей строкой будет пустая строка:

```
print(fruit[3:3])
```

Также операция среза может иметь третий параметр – шаг, т.е. полностью операция взятия среза записывается как:

value[START : STOP : STEP], где:

- value – имя переменной;
- START – это начало среза;

- STOP – это конец среза;
- STEP – это шаг хода среза.

Воспользуемся шагом среза для того, чтобы перемешать текст. Сначала будут выведены все символы, находящиеся на четных позициях, затем на нечетных.

Т	е	к	с	т		д	л	я		ш	и	ф	р	о	в	к	и
0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17

Получим две строки – символы на четных и нечетных позициях:

Четные:

Т	к	т	д	я	ш	ф	о	к
0	2	4	6	8	10	12	14	16

Нечетные:

е	с		л		и	р	в	и
1	3	5	7	9	11	13	15	17

```
text = 'Текст для шифровки'
evens = text[::2] # четные позиции
odds = text[1::2] # нечетные позиции
print(evens + odds)
```

Результатом работы программы будет:

Тктдяшфокес л ирви

Попрактикуемся:

Дан текст «Мы живы, пока мы кому-то нужны, и в малом нужны, и в великом. Покуда не знает наш дом тишины, распуганной смехом и криком...» с помощью срезов нужно вывести на экран:

1. Отдельно слово «дом»;
2. Выделяя слова срезами, соберите предложение «пока дом наш мал»;
3. Пользуясь срезами и изменяя окончания слов соберите «тишина велика, мы не знали крика».

7.1.7. Функции ord и chr

Каждый символ, в зависимости от таблицы кодировки, имеет свой номер. Например, в таблице ASCII символ «z» стоит под номером 122.

0	NUL	1	SOH	2	STX	3	ETX	4	EOT	5	ENQ	6	ACK	7	BEL
8	BS	9	HT	10	LF	11	VT	12	FF	13	CR	14	SO	15	SI
16	DLE	17	DC1	18	DC2	19	DC3	20	DC4	21	NAK	22	SYN	23	ETB
24	CAN	25	EM	26	SUB	27	ESC	28	FS	29	GS	30	RS	31	US
32	SP	33	!	34	"	35	#	36	\$	37	%	38	&	39	'
40	(	41	)	42	*	43	+	44	,	45	-	46	.	47	/
48	0	49	1	50	2	51	3	52	4	53	5	54	6	55	7
56	8	57	9	58	:	59	;	60	<	61	=	62	>	63	?
64	@	65	A	66	B	67	C	68	D	69	E	70	F	71	G
72	H	73	I	74	J	75	K	76	L	77	M	78	N	79	O
80	P	81	Q	82	R	83	S	84	T	85	U	86	V	87	W
88	X	89	Y	90	Z	91	[	92	\	93	]	94	^	95	_
96	`	97	a	98	b	99	c	100	d	101	e	102	f	103	g
104	h	105	i	106	j	107	k	108	l	109	m	110	n	111	o
112	p	113	q	114	r	115	s	116	t	117	u	118	v	119	w
120	x	121	y	122	z	123	{	124		125	}	126	~	127	DEL

Для того чтобы получить номер символа в таблице ASCII можно воспользоваться функцией ord:

```
num = ord('z')  
print(num)
```

Также существует обратная функция – чтобы преобразовать число в символ применяется функция chr:

```
num = chr(122)  
print(num)
```

Попрактикуемся:

Задание 1. Даны латинские символы «a» и «H». Сложите их номера и выведите результирующий символ на экран.

Задание 2. С помощью функции `randint` библиотеки `random` сделайте генератор случайных слов на двух языках - русский и английский. У пользователя при запуске есть выбор на каком языке он хочет получить слово. Генерируемые символы могут быть в диапазонах:

- 1) Латинские буквы верхнего регистра – от 65 по 90;
- 2) Латинские буквы нижнего регистра – от 97 по 122;
- 3) Русские буквы верхнего регистра – от 1040 по 1071;
- 4) Русские буквы нижнего регистра – от 1072 по 1103.

Длина слова может быть от 4 до 10 символов.

Задание 3. Доработайте задание 2 таким образом, чтобы программа могла генерировать абзац с 3-4 предложениями, каждое из которых состояло бы из 5-10 слов.

7.1.8. Оператор `in`

С помощью оператора `in` можно проверить принадлежит ли элемент данной последовательности, в том числе строке. Оператор `in` можно читать как «присутствует в».

Применим его на практике. Напишем программу, которая позволяет узнать присутствует ли буква «ф» в введенном с клавиатуры слове:

```
x=input()
if 'ф' in x:
    print(f'Буква "ф" присутствует в слове {x}')
else:
    print(f'Буква "ф" отсутствует в слове {x}!')
```

7.2. Строка – неизменяемый тип данных

Попробуем использовать оператор индексирования [] в левой части строки присваивания:

```
text = 'Неизменяемая строка'
text[0] = 'Т'
```

Результат работы программы:

```
TypeError: 'str' object does not support item assignment
```

То есть:

«Ошибка типа: объект «строка» не поддерживает присвоение элементов»

Подумайте:

Как же тогда получить строку «Теизменяемая строка», если в существующей строке ничего менять нельзя?

Попрактикуемся в применении операторов ветвления, циклов и строк:

Напишите программу, которая выполняет поиск в строке символа, введенного с клавиатуры. При написании кода постарайтесь продумать все возможные неуспешные сценарии или пользователь вводит символ для поиска, которого нет в слове.

Укажите в комментариях к программе все неуспешные сценарии, которые вы предусмотрели.

Пример работы программы:

Успешный сценарий	Неуспешный сценарий
Введите строку: Люблю грозу в начале мая Введите символ для поиска: о Символ находится на позиции: 8	Введите строку: Вы не ввели строку! Введите строку: Люблю грозу в начале мая Введите символ для поиска: й Такого символа в строке нет

7.3. Сырые строки

Вспоминая главу 2 «Управляющие последовательности» для решения задачи 3 «Выведите на экран путь к файлу: «C:\Users\tables\nuggets\file.txt». Разберитесь самостоятельно какая управляющая последовательность должна быть использована, чтобы вывод пути был корректным.» нужно было написать такой код:

```
print('C:\\Users\\tables\\nuggets\\file.txt')
```

Таким образом не давая в коде возникнуть управляющим последовательностям табуляции «\t» и перевода каретки «\n» с помощью последовательности «\\» была решена задача ранее

Можно было поступить по-другому, вызвав «сырую строку», т.е. строку, в которой не выполнялось вызова управляющих последовательностей, для этого надо было поставить букву r перед одинарной кавычкой строки, что говорит «raw», т.е. «сырая».

```
print(r'C:\Users\tables\nuggets\file.txt')
```

Такой подход немного проще, чем заменять вручную каждую обратную косую черту на две идущие подряд.

7.4. Методы строк

7.4.1. Поиск элемента в строке – find и index

Задачу по поиску элемента в строке можно решить куда проще, чем писать для этого циклы и условия – воспользоваться методом find или index.

Разберемся на практике в чем же разница между этими двумя методами:

```
text = 'banana'
print(text.find('a'))
print(text.index('a'))

print(text.find('o'))
print(text.index('o'))
```

Разница между ними – если искомого элемента нет, то `find` выдаст «-1», если нет при `index` – выдаст системную ошибку «`ValueError`»

Попробуйте поменять две последние строчки местами. Что изменилось?

Программа ломается на строчке «`print(text.index('o'))`», т.е. выполнение следующего за ней кода невозможно.

Эту проблему можно решить добавлением условия «если 'o' присутствует в text, то поищи его». Для этого применим оператор `in`, который переводится как «присутствует в»:

```
if 'o' in text:  
    print(text.index('o'))
```

Попрактикуемся:

Дана строка из HTML-кода сайта «`<a href="/useruploads/files/OOP2022/Mass_file_process_26.07.2022/OPOP_380302_MPPMNVT_26.07.2022.pdf" target="_blank">Открыть описание образ-ной программы</a>`» требуется получить ссылку на файл, записанный в `href`, т.е. нужно получить отдельно

«`/useruploads/files/OOP2022/Mass_file_process_26.07.2022/OPOP_380302_MPPMNVT_26.07.2022.pdf`». Для определения начала и конца взятия среза воспользуйтесь методом `index`.

Разница между методами и функциями:

Пока мы будем только использовать методы уже существующих классов, а потому пока поговорим про самое простое различие между методами и функциями. После того как мы начнем писать свои функции и изучим ООП, разница между функциями и методами будет куда больше.

Функция – независимая сущность, вызывается сама по себе, может принимать аргументы. Например, функция вывода на экран:

```
print(text)
```

Метод – зависимая сущность от класса, описывает его «поведение», вызывается у объекта. Например, в переменной `text` записана строка, т.е. это объект класса `str`, тогда с помощью «.» мы можем воспользоваться одним из методов строк – выполним поиск элемента в ней:

```
text.index('o')
```

7.4.2. Замена символов на другой - `replace`

Для задачи по замене символов используется метод `replace(X,Y)`, где `X` – строка, которую надо заменить, а `Y` – строка, на которую будет заменена прежняя.

Выполним замену всех букв «а» на «о» в тексте:

```
text = 'Апельсины, мандарины, да и сливы'
text = text.replace('a', 'o')
print(text)
```

Результатом работы программы будет:

Апельсины, мондорины, до и сливы

Чего-то не хватает, верно? Первая буква предложения не заменилась на «о», так как записана в верхнем регистре. Исправим это:

```
text = 'Апельсины, мандарины, да и сливы'
text = text.replace('a', 'o')
text = text.replace('A', 'O')
print(text)
```

И теперь будет все как задумывалось:

Опельсины, мондорины, до и сливы

Попрактикуемся:

Дано следующее зашифрованное сообщение «Лнепбблнепб гтьзу в ннепчнепле мнепя, кёгднеп весеннвый», ниже приведена таблица зашифрованных комбинаций символов в той последовательности, в которой проходило шифрование. С помощью метода `replace` восстановите изначальный текст.

Этап шифровки	Изначальная строка	Зашифрованная строка
1	и	вы
2	о	ё
3	рё	ть
4	ю	аб
5	а	неп

7.4.3. Разделение строки на подстроки - `split`

Строку можно разделить на несколько подстрок по какому-либо символу или строке с помощью метода `split()`.

Например, есть строка «22 33 4.5 -1 0», для того чтобы разделить числа на отдельные строки по пробелу можно написать:

```
text = '22 33 4.5 -1 0'
print(text.split())
```

Таким образом, мы получили список из строк чисел.

```
['22', '33', '4.5', '-1', '0']
```

С элементами списка можно работать также как и с элементами строки, т.е. можно применять оператор индексирования для обращения к отдельному элементу списка.

Если нужно разделить по пробелу, то можно писать как `split()`, так и `split(' ')`, передавая символ пробела, но он уже стоит по умолчанию.

Попрактикуемся:

Задание 1. Требуется разделить строку с перечислением продуктов «арбузы, черешня, ежевика, курица» на отдельные подстроки.

На экран вывести каждую строку отдельно списком:

1. арбузы;
2. черешня;
3. ежевика;
4. курица.

Задание 2. Дан путь к файлу:

«Q:\Загрузки\Документы ОПОП\Аннотации\08.03.01_Анн_Архитектура.pdf»

используя срезы и метод деления на подстроки нужно получить путь к папке, в которой хранится этот файл. Т.е. программа должна возвращать:

«Q:\Загрузки\Документы ОПОП\Аннотации\».

Входные данные (путь к файлу)	Выходные данные (путь к папке файла)
C:\Workspace\ВУЗ\Статья\Презентация.pptx	C:\Workspace\ВУЗ\Статья\
E:\Keras\Test.py	E:\Keras\
D:\Python_projects\Renpy\module\include\pygame.pxd	D:\Python_projects\Renpy\module\include\

7.4.4. Подсчет количества элементов – count

Для некоторых задач требуется подсчитать сколько одинаковых элементов находится в строке, это можно осуществить с помощью метода count.

Подсчитаем количество букв «е» в слове «ежевика»:

```
berry = 'ежевика'  
e_number = berry.count('e')  
print(f'Букв "е" всего: {e_number}')
```

На экран будет выведено:

```
Букв "е" всего: 2
```

Разберем пример посложнее – дана строка «черника, ежевика, арбузы, апельсины, а ещё и сливы». Подсчитайте сколько в ней букв «а», «е» и «и».

```
text = 'черника, ежевика, арбузы, апельсины, а ещё и сливы'  
print(f'Букв А в строке: {text.count("а")}')  
print(f'Букв Е в строке: {text.count("е")}')  
print(f'Букв И в строке: {text.count("и")}')
```

Результат работы программы:

```
Букв А в строке: 5  
Букв Е в строке: 5  
Букв И в строке: 5
```

Попрактикуемся:

Задание 1. Дана строка «халапеньо, длинношеее, хворост, змеед, Ыгыатта, воспользовавшемся, апельсин». Требуется определить наибольшее количество букв «е» в одном слове и вывести одно из таких слов.

Задание 2. Попробуйте с помощью кода из Задания 1 найти слово с наибольшим количеством букв «ё». Ваша программа не должна при этом молчать или выводить неверное значение.

7.4.5. Некоторые другие упрощающие работу методы

У строк есть множество других методов, которые упрощают работу с ними, в таблице ниже будут приведены некоторые из них:

Метод	Его значение
lower()	Переводит всю строку в нижний регистр
upper()	Переводит всю строку в верхний регистр
islower()	Записана ли вся строка в нижний регистр
isupper()	Записана ли вся строка в верхний регистр
capitalize()	Первая буква строки будет в верхнем регистре, остальные – в нижнем
strip()	Удаляет лишние пробелы в начале и конце строки. Т.е. строка « слово » станет «слово» « несколько слов » станет «несколько слов»

Попрактикуемся:

Шифр Цезаря или же шифр сдвига является простым способом шифрования, который включает в себя сдвиг каждого символа строки на определенное количество позиций в кодировке. Сдвиг циклический – т.е. сдвиг «Я» на 1 позицию возвратит букву «А».

Шифр сдвига не является надежным шифром, в нынешнее время он применяется для различных несложных шифровок в качестве шуток или отсылок, например:

- В фильме «Космическая Одиссея 2001 года» применяется шифр Цезаря к имени ИИ корабля «HAL», если буквы сместить на -1 получится «IBM», создатели сделали отсылку к одному из крупнейших производителей и поставщиков аппаратного и программного обеспечения.
- Также шифр ROT13 («ротация/сдвиг на 13 символов») используется в интернет-форумах как инструмент сокрытия спойлеров, решений загадок и оскорбительных шуток от случайных взглядов.

Требуется написать программу, которая шифрует введенное сообщение на заданное пользователем количество символов (пробел сдвигать не надо).

К сожалению, создать сдвиг с помощью `ord` и `chr` будет проблематично – буква «ё» не идет после «е», которая стоит на позиции 1077. Следующую позицию занимает буква «ж». Буква «ё» находится после «я», на позиции 1105. Проще создать строку, в которую вы запишете все буквы русского алфавита и на основе этой строки будете совершать сдвиг. По сути, ничего не меняется – `ord` осуществляет поиск по таблице кодировки, ровно как `index` сможет найти позицию символа в вашем алфавите, а `chr`, с помощью которого можно обращаться к символу по номеру в таблице имеет такой же смысл как операция индексирования к строке-алфавиту.

Введенное значение	Вывод на экран
Введите сообщение: Пробежка утром и днём Сдвиг: 7	Зашифрованное сообщение: Цчхзлнсж ъщчху п кфму
Введите сообщение: Уднцтс рткиц рстжти Сдвиг: -4	Зашифрованное сообщение: Пайтон может многое

8. Списки

Список (list) – это упорядоченная изменяемая коллекция объектов произвольных типов (почти как строка, но список не ограничен только символами). Значения в списке называются элементами (elements или items).

Чтобы объявить список можно двумя способами. Не важно будет как вы это делаете, хотя, на мой взгляд, квадратные скобки поставить проще и быстрее:

```
inventory = list()
inventory = []
```

Можно создать список, в котором заранее есть некоторые элементы:

```
inventory = ['яблоко', 'меч', 'сапоги']
```

Список может состоять из элементов различных типов:

```
x = [22, 'пони', False]
```

Обход списка с помощью цикла

Списки можно обходить также, как и строки с помощью цикла while:

```
inv = ['яблоко', 'меч', 'сапоги']
k = 0
while k < len(inv):
    print(inv[k])
    k += 1
```

В качестве примера, решим следующую задачу:

Дан список целых чисел [12, -5, 60, 123, 1, 0, -4] с помощью операторов ветвления и циклов определите максимальное значение в списке.

```
# Задаю список
num_list = [12, -5, 60, 123, 1, 0, -4]

# Создаю переменную, которая будет запоминать
# максимальное найденное значение

# По умолчанию его можно задать равному нулевому элементу
max_num = num_list[0]
```

```

# Создаем переменную цикла,
# т.к. максимальное значение по умолчанию
# равно нулевому элементу, то проверять его не надо,
# начинаем с 1
k = 1

while k < len(num_list):
    # Запишем просматриваемый элемент
    # в отдельную переменную для удобства
    elem = num_list[k]

    # Если элемент больше текущего
    # максимального, то он теперь максимальный
    if elem > max_num:
        max_num = elem
    k += 1
print(f'Максимальное значение в списке: {max_num}')

```

Результат работы программы:

Максимальное значение в списке: 123

Попрактикуемся:

Задание 1. Дан список целых чисел [8, 76, 56, -100, 11, 66] с помощью операторов ветвления и циклов определите минимальное значение в списке.

Задание 2. Дан список целых чисел [31, -2, -5, 66, 81, 93] с помощью оператора цикла определите сумму чисел, находящихся в списке.

Задание 3. Задан список названий и цен на мороженое в формате:

<наименование мороженого 1>, <стоимость мороженого 1>, ...,
 <наименование мороженого N>, <стоимость мороженого N>

Список наименований мороженого и цен на него представлен ниже:

```

['Бодрая корова', 55.99, 'Клубничный слон', 99.40, 'Русский
шарик', 67.7, 'Сладкая сказка', 88.50, 'Сложный выбор', 140.55,
'Березовая роща', 40.5]

```

В Python визуально можно отобразить ручную запись значений в подобные списку структуры данных удобнее, с использованием перевода на новую строку и табуляции:

```

icecream_list = [
    'Бодрая корова', 55.99,
    'Клубничный слон', 99.40,

```

```
'Русский шарик', 67.7,  
'Сладкая сказка', 88.50,  
'Сложный выбор', 140.55,  
'Березовая роща', 40.5  
]
```

Требуется написать программу, которая выберет мороженое, которое будет ближе к общей средней цене из представленных. Вывести на экран среднюю цену на мороженое, наименование и цену самого ближнего мороженого к этой цене.

Для того, чтобы посчитать самое ближнее к среднему мороженое, нужно воспользоваться взятием по модулю – функция `abs`.

8.1. Вложенные списки

Также список может содержать в себе другие списки, например:

```
num_list = [[1, 22, -1], [44, 5], [-0.5, -2, 3]]
```

Давайте зададимся вопросом – какова длина списка `num_list`? Т.е. сколько в нём сейчас элементов? Правильный ответ – 3, вы можете это проверить с помощью `len`:

```
print(len(num_list))
```

В списке всего 3 элемента – это три списка:

1. [1, 22, -1];
2. [44, 5];
3. [-0.5, -2, 3].

Правильно будет сказать, что список `num_list` содержит в себе три вложенных списка. **Вложенный список** – это такой список, который хранится в другом списке.

Использование вложенных списков может быть удобным решением для организации простой базы данных (БД). Например, для записи ФИО и дней рождений друзей в записной книжке, можно создать список:

```
friends_data = [  
    ['Маркелов А.Е.', '07.06.1984'],  
    ['Александрова Н.И.', '03.01.1996'],  
    ['Носов Т.И.', '29.12.1997']  
]
```

Напишем код, в котором определим у какого из друзей день рождения 29 декабря, т.е. 29.12:

```
friends_data = [  
    ['Маркелов А.Е.', '07.06.1984'],  
    ['Александрова Н.И.', '03.01.1996'],  
    ['Носов Т.И.', '29.12.1997']  
]  
  
k = 0  
while k < len(friends_data):  
    # Данные об одном друге  
    friend = friends_data[k]  
    # Затем получаем дату рождения  
    birthday = friend[1]  
    if '29.12' in birthday:  
        print(f'У {friend[0]} день рождения 29 декабря!')  
    k += 1
```

Можно написать по-другому:

```
friends_data = [  
    ['Маркелов А.Е.', '07.06.1984'],  
    ['Александрова Н.И.', '03.01.1996'],  
    ['Носов Т.И.', '29.12.1997']  
]  
  
k = 0  
while k < len(friends_data):  
    # Получаем сразу дату рождения двойным оператором  
    индексирования  
    birthday = friends_data[k][1]  
    if '29.12' in birthday:  
        # Также и имя друга:  
        print(f'У friends_data[k][0] день рождения 29  
декабря!')  
    k += 1
```

Попрактикуемся:

Задание 1. Дан список [[11, 56, 93], [76, 93], [88, -1, 3, 4], [1]], требуется вычислить сумму списка, т.е. общую сумму всех вложенных списков.

Задание 2. С помощью вложенных списков была организована БД ветеринарной клиники, каждая запись хранится в формате:

[<ФИО владельца>, <Кличка питомца>, <Дата и время приема>]

БД заполнена вручную:

```
vet_db = [  
    ['Ломоносова Н.В.', 'Тима', '11.01.2022 12:50:00'],  
    ['Линяев А.Г.', 'Мурзик', '13.05.2021 10:15:00'],  
    ['Ширяева Н.С.', 'Капа', '01.03.2023 16:55:00'],  
    ['Вознесенский Т.П.', 'Панда', '21.04.2022 22:00:00'],  
    ['Дубовицкая Т.Н.', 'Мора', '09.03.2023 11:00:00'],  
    ['Большакова А.М.', 'Ильфас', '11.05.2021 17:43:00'],  
    ['Ларионов С.М.', 'Геля', '08.07.2022 15:40:00'],  
    ['Арбузов К.В.', 'Юккер', '12.06.2020 12:35:00'],  
    ['Даниленко С.С.', 'Кузя', '02.07.2022 08:25:00'],  
    ['Николаев Е.Р.', 'Портос', '07.12.2021 11:05:00'],  
    ['Шумелова О.К.', 'Мотя', '24.04.2023 15:50:00'],  
]
```

Работая с этой БД, требуется выполнить задания 2.1 – 2.3.

Задание 2.1. Напишите программу, которая по кличке, введенной с клавиатуры, выводит на экран ФИО владельца питомца и на какой день и время назначен прием.

Ввод пользователя	Результат работы программы
Введите кличку питомца: Панда	Владельцем питомца является: Вознесенский Т.П. Прием назначен на: 21.04.2022 22:00:00
Введите кличку питомца: Тимофей	Питомца с такой кличкой нет в БД

Задание 2.2. Напишите программу, которая выводит на экран все записи о приемах за определенный год. Например:

Ввод пользователя	Результат работы программы
Введите год, за который нужно отобразить данные: 2021	Найдено записей: 3 Запись 1: ФИО: Линяев А.Г. Кличка питомца: Мурзик Дата приема: 13.05.2021 Время приема: 10:15:00 Запись 2: ФИО: Большакова А.М. Кличка питомца: Ильфас

	Дата приема: 11.05.2021 Время приема: 17:43:00 Запись 3: ФИО: Николаев Е.Р. Кличка питомца: Портос Дата приема: 07.12.2021 Время приема: 11:05:00
Введите год, за который нужно отобразить данные: 2019	Найдено записей: 0 Данные за 2019 год отсутствуют

Задание 2.3. Напишите программу, которая выводит на экран все записи, время приема которых происходит в 11 часов.

8.2. Список – изменяемый тип данных

В отличие от строк, элементы списка можно изменять, используя оператор индексирования [] (обращаясь к ним по индексу):

```
num_list = [22, 44, 55, 11]
num_list[1] = 125
num_list[2] = 'печенье'
num_list[0] = True
print(num_list)
```

Оператор индексирования у списка работает по тем же правилам, что и индексирование у строк:

- индексом может быть любое целочисленное значение;
- индекс может быть отрицательным, тогда отсчет начинается с конца списка;
- обращение к индексу, которое заходит за границы списка вызывает ошибку индексации – «*IndexError*».

8.3. Методы списков

8.3.1. Добавление элемента в список – метод append

Метод `append` позволяет добавить новый элемент в конец списка

```
x = [] # создание пустого список
x.append(23) # добавление числа
x.append('пирог') # добавление текста
x.append([11, 23, 433]) # добавление вложенного списка
print(x)
```

Попрактикуемся:

Задание 1. Программа считывает с клавиатуры целые числа (*int*), пока не будет введен ноль. Каждое значение добавляется в список `numbers_list`. После окончания ввода, изменить все значения на 10 больше вводимых и вывести список на экран.

Ввод значений	Вывод на экран
22 -1 33 100 0	[32, 9, 43, 110]

Задание 2. Строка задаётся с клавиатуры, в которой перечисляются слова через запятую и пробел. Требуется определить наибольшее количество букв «о» в одном слове, выписать все слова с максимальным количеством букв «о» в отдельный список и вывести их все на экран.

Ввод пользователя	Результат работы программы
арматура, структура, кот, колоратура, нарезка, довод, терка, овод, щепетильный, обзор, машина, монитор, шёлк, виноград, гусеница, арбуз, бабочка, слива, мотор	Наибольшее количество букв "о" в слове: 2 Слова с наибольшим количеством "о": 1. колоратура; 2. довод;

	3. овод; 4. обзор; 5. монитор; 6. мотор.
шелуха, флегма, водород, пиявка, водоросли, долото, лото, заноза, золотой, колосок, робот, стихи, семечки, греча, грибы, рис	Наибольшее количество букв "о" в слове: 3 Слова с наибольшим количеством "о": 1. водород; 2. водоросли; 3. долото; 4. золотой; 5. колосок.
кокон, магазин, опера, опора, лестница, умора, окно, красный, карандаш, кошка, усы, ручка, банка, русло	Наибольшее количество букв "о" в слове: 2 Слова с наибольшим количеством "о": 1. кокон; 2. опора; 3. окно.
ересь, непримиримый, лёд, палка, решение, ёлка, деревья	В строке отсутствуют слова с буквой "о"

Задание 3. Требуется написать программу, которая выполняет разбор слова – выводит из каких букв оно состоит и сколько этих букв в нём.

Для упрощения организации структуры данных можете воспользоваться связанными списками. Имеется ввиду, что списки «связаны» между собой одним индексом, т.е. под одним списке могут находиться данные о уникальных буквах, а в другом сколько таких букв в слове.

Ввод пользователя	Результат работы программы
Введите слово для разбора: слово	Слово "слово" состоит из 4 уникальных букв: 1. с – 1; 2. л – 1; 3. о – 2;

	4. в - 1.
Введите слово для разбора: колосок	Слово "колосок" состоит из 4 уникальных букв: 1. к - 2; 2. о - 3; 3. л - 1; 4. с - 1.

8.3.2. Расширение списка – метод extend

Метод расширения списка позволяет добавить в конец одного списка данные из другого.

Разберем метод extend на примере - два товарных поезда были на остановке, один вёз мясо, рыбу и сыр, второй вёз яблоки, огурцы и груши. Локомотив (передняя часть поезда) второго поезда сломался, было решено добавить товарные вагоны с яблоками, огурцами и грушами в конец первого поезда.

Смоделируем эту ситуацию:

```
train_1 = ['мясо', 'рыба', 'сыр']
train_2 = ['яблоки', 'огурцы', 'груши']
train_1.extend(train_2)
print(train_1)
```

Результат работы программы:

```
['мясо', 'рыба', 'сыр', 'яблоки', 'огурцы', 'груши']
```

Отметим важный момент, который происходил в методах append и extend – подобные методы изменяют элемент, над которыми производятся, чего не происходило при вызове методов строк. Т.е. не нужно писать:

```
train_1 = train_1.extend(train_2)
```

Так как он уже изменяет данные в train_1 при вызове extend:

```
train_1.extend(train_2)
```

8.3.3. Удаление элемента из списка по индексу – метод pop

Для того чтобы удалить элемент из списка следует воспользоваться методом pop, в качестве аргумента необходимо передать индекс элемента, который должен быть удален.

Предположим, что первый поезд из предыдущей задачи оставил на какой-то остановке весь груз с рыбой, который находится по индексу 1:

```
train = ['мясо', 'рыба', 'сыр']
train.pop(1)
print(train)
```

Программа выведет:

```
['мясо', 'сыр']
```

Обратите внимание, что при удалении элемента «рыба», элемент «сыр» переместился на индекс 1.

мясо	рыба	сыр	->	мясо	сыр
0	1	2		0	1

Таким образом, при удалении нескольких элементов вы должны следить, чтобы переменная цикла не «перепрыгивала» через элементы. Разберем пример:

Из списка [11, 52, -48, 11, 65, -18, 20, 11, 7, 46, -58, 11, 99, 74, 53, 11, 11, 11, 11, -84, -11, -84, 11] требуется удалить все числа 11. Напишем следующий код:

```
num_list = [11, 52, -48, 11, 65, -18, 20, 11, 7, 46, -58, 11, 99, 74, 53, 11, 11, 11, 11, -84, -11, -84, 11]

k = 0
while k < len(num_list):
    if num_list[k]==11:
        num_list.pop(k)
    k += 1
print(num_list)
```

Попробуйте запустить такой код, программа должна вывести:

[52, -48, 65, -18, 20, 7, 46, -58, 99, 74, 53, 11, 11, -84, -11, -84]

Программа не удалила все элементы, как же так? Вспомните про сдвиг при удалении элементов, о котором мы ранее говорили. Числа 11 остались только там, где они повторялись несколько раз подряд. Рассмотрим более простой пример, интересующие нас ячейки переменной цикла k будут отмечены красным цветом:

Код найдет число 11 на первой позиции и удалит его

Список	4	11	11	11
Переменная цикла k	0	1	2	3

Переменная цикла k станет равна 2, а список станет выглядеть так:

Список	4	11	11
Переменная цикла k	0	1	2

Так как переменная цикла теперь 2, то он увидит снова число 11 и удалит его со второй позиции, получится такой список:

Список	4	11
Переменная цикла k	0	1

Таким образом, один элемент всё равно останется. Для того чтобы решить данную проблему надо сделать так, чтобы при нахождении числа 11 на этой итерации переменная цикла не изменилась, или же совокупное изменение было равно нулю, т.е. можно сделать, чтобы при удалении k становилась на 1 меньше, а затем по умолчанию на 1 больше, т.е.:

```
num_list = [11, 52, -48, 11, 65, -18, 20, 11, 7, 46, -58, 11, 99, 74, 53, 11, 11, 11, 11, -84, -11, -84, 11]
```

```
k = 0
while k < len(num_list):
    if num_list[k]==11:
        num_list.pop(k)
```

```
        k -= 1 # Всё что нужно было добавить!  
    k += 1  
print(num_list)
```

И теперь код будет решать задачу верно:

```
[52, -48, 65, -18, 20, 7, 46, -58, 99, 74, 53, -84, -11, -84]
```

Попрактикуемся:

Задание 1. Дан список чисел [11, 0, 2, -1, 55, 100, 0.5, 68, 223], удалите из него первые три минимальных значения.

Задание 2. Из строки «найденых, кактус, водоросли, фикус, манго, макака, компания, экономика, нагороженные, припрятавшему, прусско-австрийский, сверхсветовыми, маффин, просвещающую, зацкливавшими, маслиновому, разогнувшейся, прозвонился» требуется получить список слов, а затем исключить из него слова, содержащие букву «о».

Задание 3. Дан список целых чисел [1, -11, 100, 2, 59, 70, -94, -71, -69, 11, 39, 5, -56, 47, 96, 2, -99, -6], должны остаться только числа, которые идут за числами 2 и 11.

Результатом работы программы должен быть список:

```
[59, 39, -99]
```

Задание 4. Из списка [6, 82, 44, -1, 5, 33, 28, 36, 0] удалите элементы 33, 5, -1, 28. Без метода index.

8.3.4. Поиск элемента в списке – метод index

Метод `index`, как и у строк позволяет определить по какому индексу находится элемент.

Определим какой по порядку вагон с сыром:

```
train = ['мясо', 'рыба', 'сыр']
cheese_index = train.index('сыр')
print(f'Вагон с сыром находится под индексом: {cheese_index}')
```

Программа выведет:

Вагон с сыром находится под индексом: 2

Теперь разберемся в тонкостях работы метода. Что если в списке несколько записей «сыр»? Какой индекс будет выведен? Метод `index` находит **первое вхождение** указанного значения, убедимся в этом.

Расположим «сыр» по индексам 2 и 5:

```
train = ['мясо', 'рыба', 'сыр', 'мясо', 'рыба', 'сыр']
cheese_index = train.index('сыр')
print(f'Вагон с сыром находится под индексом: {cheese_index}')
```

Программа выведет такой же результат, как и ранее:

Вагон с сыром находится под индексом: 2

Попрактикуйтесь:

Напишите программу, которая выводит на экран индекс элемента, введенного с клавиатуры, из списка «мясо, сыр, рыба».

8.3.5. Поверхностная копия списка – метод `copy`

На самом деле, когда мы создаём переменную – мы привязываем область данных к определённому имени переменной. Понимание этого важно, разберём пример:

```
train_1 = ['мясо', 'рыба', 'сыр']
train_2 = train_1

train_2.append('булка')

print(f'Первый поезд содержит вагоны:\n{train_1}\n')
print(f'Второй поезд содержит вагоны:\n{train_2}')
```

Результатом работы программы будет:

```
Первый поезд содержит вагоны:
['мясо', 'рыба', 'сыр', 'булка']

Второй поезд содержит вагоны:
['мясо', 'рыба', 'сыр', 'булка']
```

Получилось, что в `train_1` точно также добавился вагон с булкой.

Почему?

На второй строчке кода выше «`train_2 = train_1`» мы связали область данных `train_1` с новым именем переменной – `train_2`, таким образом у одной области данных теперь два имени. Чтобы избежать нежелательного изменения данных нужно применять метод поверхностного копирования – `copy`.

Доработаем пример выше с помощью метода `copy`:

```
train_1 = ['мясо', 'рыба', 'сыр']
train_2 = train_1.copy()

train_2.append('булка')

print(f'Первый поезд содержит вагоны:\n{train_1}\n')
print(f'Второй поезд содержит вагоны:\n{train_2}')
```

Теперь результат работы программы будет корректный:

Первый поезд содержит вагоны:

```
['мясо', 'рыба', 'сыр']
```

Второй поезд содержит вагоны:

```
['мясо', 'рыба', 'сыр', 'булка']
```

Попробуйте такой код:

```
train_1 = ['мясо', 'рыба', 'сыр', []]  
train_2 = train_1.copy()  
  
train_2.append('булка')  
train_1[3].append(2)  
  
print(f'Первый поезд содержит вагоны:\n{train_1}\n')  
print(f'Второй поезд содержит вагоны:\n{train_2}')
```

8.3.6. Подсчёт количества элементов – метод count

Для подсчёта количества элементов, ровно как и у строк, можно воспользоваться методом count.

Обозначим каждый вагон с содержимым за номер:

Номер	Тип вагона
1	Мясо
2	Рыба
3	Сыр
4	Яблоки

Тогда запись о том, что поезд состоит из вагонов «мясо» - «рыба» - «сыр» - «яблоки» - «мясо» - «мясо» - «рыба» - «яблоки» - «сыр» - «мясо» можно сократить до списка с номерами:

```
train = [1, 2, 3, 4, 1, 1, 2, 4, 3, 1]
```


С помощью метода count подсчитаем сколько мяса, рыбы, сыра и яблок перевозит поезд:

```
meat_count = train.count(1)
fish_count = train.count(2)
cheese_count = train.count(3)
apples_count = train.count(4)

print(f'Вагонов с мясом: {meat_count}')
print(f'Вагонов с рыбой: {fish_count}')
print(f'Вагонов с сыром: {cheese_count}')
print(f'Вагонов с яблоками: {apples_count}')
```

Такой код довольно неэффективный, если предположим, что у нас много различных вагонов, не будем же мы выписывать на каждую строчку новый подсчет и новый вывод данных.

Попрактикуемся:

Задание 1. Расширим задачу выше - дежурный остановки обязан передавать данные о вагонах. Для простоты он договорился с оператором о нумерации:

Номер	1	2	3	4	5
Тип вагона	Мясо	Рыба	Сыр	Яблоки	Лук

Номер	6	7	8	9	10
Тип вагона	Помидоры	Кленовый сироп	Черный чай	Макароны	Консервы

Дежурный передал следующие данные:

5, 1, 7, 10, 2, 3, 5, 5, 7, 2, 6, 3, 1, 4, 3, 6, 8, 4, 7, 7, 5, 3, 2, 9, 5, 9, 9, 7, 9, 9,
6, 2, 10, 2, 8, 3, 2, 1, 7, 8, 8, 2, 7, 10, 5, 3, 9, 7, 1, 6

Создайте список, в котором перечисляются типы вагонов «*train_types*», затем создайте список с данными, переданными дежурным «*attendant_data*». После этого в цикле переберите все наименования товаров и подсчитайте сколько таких вагонов в поезде.

Если вы всё сделали правильно, то теперь можно добавить одиннадцатый тип вагона – «апельсины» и если в список «*attendant_data*» добавить в случайные места несколько чисел «1», то программа будет корректно подсчитывать и новый тип вагонов.

Задание 2. На вход программе подаются две строки – первая содержит последовательность из слов, а вторая состоит из чисел, которые соответствуют слову под номером, эта строка отражает частоту встречаемости слов в некотором тексте. Требуется вывести на экран список из слов, отсортированный по встречаемости слова. Наиболее встречаемый стоит на первом месте.

В качестве сортировки списка можете использовать самый простой способ – сортировку пузырьком.

Ввод пользователя	Результат работы программы
Строка слов: чечевица, киви, фарватер, круг Встречаемость слов в тексте: 1, 2, 3, 4, 4, 4, 4, 2, 2, 3	Отсортированный список по встречаемости слов: ['круг', 'киви', 'фарватер', 'чечевица']
Строка слов: дедуцирующая, разругавшаяся, воззавшую, обвиваешься, почасной, нефтяником, тафт, мнимобольного, засухоустойчивого, ятрохимия, сдергивавшей, трогать, перепортившимися, вытациваемыми, остригавшимися, остросовременное, облеплявшимся, бристольтских, ретранслирующийся, оплакиваться	Отсортированный список по встречаемости слов: ['бристольтских', 'ятрохимия', 'дедуцирующая', 'мнимобольного', 'засухоустойчивого', 'облеплявшимся', 'оплакиваться', 'разругавшаяся', 'почасной', 'сдергивавшей', 'перепортившимися', 'вытациваемыми', 'воззавшую', 'обвиваешься', 'нефтяником', 'остригавшимися',

Встречаемость слов в тексте: 20, 18, 9, 11, 9, 8, 17, 13, 10, 5, 20, 19, 1, 4, 13, 14, 17, 21, 10, 8, 5, 18, 18, 20, 1, 18, 14, 6, 8, 18, 11, 17, 2, 15, 9, 2, 10, 3, 1, 10	'ретранслирующийся', 'тафт', 'трогать', 'остросовременное']
--	--

Так как подходы к решению могут быть разные, т.е. сортировка слов с одинаковой встречаемостью может отличаться, то для тестов приложим также их встречаемость:

Тест 1:

[4, 3, 2, 1]

Тест 2:

[5, 4, 3, 3, 3, 3, 3, 2, 2, 2, 2, 2, 1, 1, 1, 1, 1, 0, 0, 0]

8.4. Срезы списков

У списков, как и у строк, можно брать срезы:

```
num_list = [22, 54, 11, 77]
print(num_list[0:2])
print(num_list[:3])
print(num_list[::2])
```

Но это ещё не всё! Так как списки – изменяемый тип данных, то срезы можно использовать и в левой части оператора присваивания, т.е.

использовать срезы для изменения сразу нескольких элементов списка:

```
num_list = [22, 54, 11, 77]
num_list[:3] = [1, -1, 0]
print(num_list)
```

На экран будет выведено:

```
[1, -1, 0, 77]
```

Попрактикуйтесь:

Задание 1. Проверьте, что случится, если в правой части будет передано меньше элементов, чем взятый срез? Что если больше?

Задание 2. Вывести на экран элементы, которые находятся между словами, у которых больше всего букв «а» и «е». Если слов, в которых таких максимальных букв несколько, то выбирается первое по порядку. Если слово содержит одновременно и максимальное количество букв «а», и «е», то программа должна вернуть пустой список.

Ввод пользователя	Результат работы программы
Введите строку из: элизиум, дерево, гусеница, бабочка, апостроф, отдых, гамак, пшеница, перегородка	Результирующий список равен: ['гусеница']
Введите строку: рыба-меч, кокос, арбуз, пеликан, листва, яблоки, ананас, водоросли, сухофрукты, океан, переехал, моллюски, вишня, креветки, баклан	Результирующий список равен: ['водоросли', 'сухофрукты', 'океан']

Введите строку: разорвавшемуся, тотем, жёлудь, дуб, корень, кусты	Результирующий список равен: []
---	------------------------------------

Пояснение к первому тесту, промежуточный вывод на экран:

Максимальное количество букв "а" содержит слово "бабочка" по индексу 3

Максимальное количество букв "е" содержит слово "дерево" по индексу 1

Срез будет взят от 2 («дерево» не входит) до 3

8.5. Связь между списками и строками

8.5.1. Преобразование строки в список – list и split

Несмотря на то, что строка – последовательность символьных значений, а список – последовательность произвольных значений, строка – это не список символов. Изменить напрямую элемент строки нельзя, в отличие от списка. У списка есть методы, позволяющие добавлять и удалять элементы, у строки – нет.

В зависимости от выполняемых задач, вам понадобится переводить строки в списки, а списки – в строки.

Для перевода строки в список можно воспользоваться функцией list:

```
text = 'текст-строка'
text_to_list = list(text)
print(text_to_list)
```

Выполнив такой код, на экран будет выведен список, в котором каждый элемент – символ из строки:

```
['т', 'е', 'к', 'с', 'т', '-', 'с', 'т', 'р', 'о', 'к', 'а']
```

Для разбиения на отдельные слова в списке не забывайте про использование метода split.

8.5.2. Преобразования списка в строку – join

Также для преобразования списка, состоящего из строк, в одну большую строку, можно воспользоваться методом строк join, который не был упомянут в разделе «Методы строк».

Создадим список, в котором хранится несколько элементов-строк «мясо», «рыба», «сыр». Нужно получить одну строку, в которой эти элементы записаны через запятую. Для начала сделаем это с помощью цикла while:

```
prod_list = ['мясо', 'рыба', 'сыр']
prod_text = ''
k = 0
while k < len(prod_list):
    prod_text += prod_list[k] + ','
    k += 1
```

```
prod_text = prod_text[:-1] # Используя срез, чтобы убрать лишнюю запятую
print(prod_text)
```

Выполнив такой код, получим:

мясо,рыба, сыр

Конкретно для объединения элементов в одну строку нужно было написать 6 строк, но с помощью метода join это можно записать в одну строчку:

```
prod_list = ['мясо', 'рыба', 'сыр']
prod_text = ','.join(prod_list)
print(prod_text)
```

Результатом выполнения программы такая же строка:

мясо,рыба, сыр

Формально описать структуру метода join (объединения) можно как:

<строка-разделитель>.join(<список>)

Попрактикуемся:

Задание 1. Ранее было уже предложено решить задание ниже, сборка пути к папке до этого выполнялась с помощью цикла while и конкатенации строк, теперь же это вы можете сделать в одну строчку с помощью метода join:

«Q:\Загрузки\Документы ОПОП\Аннотации\08.03.01_Анн_Архитектура.pdf» используя срезы и метод разделения на подстроки нужно получить путь к папке, в которой хранится этот файл. Т.е. программа должна возвращать: «Q:\Загрузки\Документы ОПОП\Аннотации\».

Входные данные (путь к файлу)	Выходные данные (путь к папке файла)
C:\Workspace\ВУЗ\Статья\Презентация.pptx	C:\Workspace\ВУЗ\Статья\
E:\Keras\Test.py	E:\Keras\

D:\Python_projects\Renpy\module\include\pygame.pxd	D:\Python_projects\Renpy\module\include\
--	--

Задание 2. Различные книги по Пайтону хранятся в разделе «Books\Python» вперемешку, вне зависимости от направленности пособия. Ранее пользователь добавил в начало каждой книги метку о чем она, например, если она относится к искусственному интеллекту, то вначале будет метка «(ИИ)». Гарантируется, что метка содержит не больше одного слова, заключена в круглые скобки и отделяется пробелом.

Применяя срезы и метод join требуется написать программу, которая примет существующие пути «Books\Python» и заменит на «Книги\<наименование метки>\<название книги без метки>».

Ввод пользователя	Результат работы программы
Путь книги для изменения: Q:\Books\Python\ (ИИ) ИИ и компьютерное зрение.pdf	Измененный путь книги: Q:\Книги\ИИ\ИИ и компьютерное зрение.pdf
Путь книги для изменения: Z:\MyDocuments\Books\Python\ (Паттерны) Паттерны разработки на Python TDD, DDD и событийно-ориентированная архитектура.pdf	Измененный путь книги: Z:\MyDocuments\Книги\Паттерны\Паттерны разработки на Python TDD, DDD и событийно-ориентированная архитектура.pdf
Путь книги для изменения: E:\Python\Books\Python\ (Основы) Python для чайников.pdf	Измененный путь книги: E:\Python\Книги\Основы\Python для чайников.pdf

8.6. Алгоритмы сортировки списков

8.6.1. Сортировка пузырьком – Bubble sort

8.6.2. Сортировка вставками – Insertion sort

8.6.3. Быстрая сортировка – Quick sort

8.7. Операции над списками

- 8.7.1. Поиск максимума и минимума – max и min
- 8.7.2. Сумма всех элементов – sum
- 8.7.3. Сортировка списка – sort
- 8.7.4. Объединение списков

9. Цикл for

Ранее мы использовали для работы с циклами ключевое слово while, в таких циклах условием завершения будет являться какое-либо условие. Но что, если нам нужно просто пройти по списку или строке, перебрав все её элементы или же просто отсчитать от 0 до 11? Для таких случаев также существует цикл for. Сравним работу while и for.

Задача – имеется список продуктов products, нужно вывести все элементы списка на экран.

С помощью цикла while это выполнялось бы с помощью переменной цикла, которая бы менялась на каждой итерации, а с помощью индексирования выводился элемент на экран:

```
products = ['apples', 'fish', 'meat']

k = 0
while k < len(products):
    product = products[k]
    print(product)
    k += 1
```

С помощью же цикла for можно сразу получать элементы списка-продукты, без переменной цикла и обращения по индексу:

```
products = ['apples', 'fish', 'meat']

for product in products:
    print(product)
```

Вышло куда лаконичнее, верно? Зачастую, когда в цикле нужно просто перебрать последовательно элементы, цикл for использовать куда удобнее. Требуется отдельно отметить оператор «in», с которым мы уже работали при использовании операторов ветвления, если нужно было проверить принадлежит ли некоторый элемент списку. Всю строчку можно прочитать как «Для продукта в продуктах сделай то-то».

```
products = ['apples', 'fish', 'meat']

k=0
for product in products:
    print(k,product)
    k+=1
```

Попрактикуемся:

Задание 1. В списке, задаваемого числами, введенными с клавиатуры одной строкой и разделенными через запятую, с помощью цикла for определите сумму и минимальное значение и выведите их на экран.

Результатом работы программы должно быть:

Ввод пользователя	Результат работы программы
Введите числа через запятую: 145, 96, -18, -68, 113, 63, -63, -21, 20, 35	Сумма: 302 Минимальное значение: -68
Введите числа через запятую: 102, -50, 143, 103, 5, -16, 84, 68, 92, -20	Сумма: 511 Минимальное значение: -50
Введите числа через запятую: 102, 130, 65, 86, 10, -63, 113, -54, 21, -9, 97, -8, 41, 65, 112, -52, -35, 65, -50, 50	Сумма: 686 Минимальное значение: -63

Задание 2. С помощью цикла for обработайте входящую строку несколькими способами – в качестве первого ответа программа должна вывести входную строку без букв «о» и «а», вторым ответом программа должна вывести только символы, находящиеся на четных позициях.

Ввод пользователя	Результаты работы программы
Введите слово: абрикос	Строка без «о» и «а»: брикс Строка из четных позиций: аркс
Введите слово: лошадка	Строка без «о» и «а»: лшдк Строка из четных позиций: лшда

Задание 3. Требуется написать программу, которая будет подсчитывать сумму баллов каждого из участников команды.

На вход подаются строки – в них перечислены наименование команды и баллы команды, но из-за ошибки, наименование команды располагается в случайном месте. При вводе «0», как отдельной строки, программа должна завершать считывание данных.

Требуется найти сумму баллов каждой из команд, вывести их в формате пар «команда» - «сумма баллов», а также назвать победителя – команду с самой большой суммой баллов.

Для того чтобы работать с отдельными словами или числами в строке их надо разбить на подстроки и перебирать с помощью цикла for. При переборе значений строки вам нужно будет понять является ли подстрока числом или нет, для этого есть в Python метод строк `isdigit()`. Например, если вызвать `'25'.isdigit()`, то результатом будет `True`, а если `'птица'.isdigit()` – `False`.

Пояснение к заданию:

Сначала запомните все данные о командах в список `teams_db` с помощью цикла `while`, затем с помощью цикла `for` перебирайте данные о каждой команде и определяйте тип данных с помощью `isdigit()`.

Ввод пользователя	Результат работы программы
Введите данные о командах: 1, 22, 100, 50, 33, Оплот 68, 59, Медуза, 76, 81, 22 41, Молот, 40, 95, 16, 47, 8, 39 Наковальня, 7, 32, 8, 5, 10 0	Данные по каждой команде: Оплот – 206 Медуза – 306 Молот – 286 Наковальня – 62 Победителем является команда: Медуза

9.1. Изменение параметра цикла с помощью range

Если нужно просто изменять параметр цикла, как это происходило бы с помощью while:

```
k = 0
while k < 5:
    print(k)
    k += 1
```

В цикле for сказать «изменяй параметр цикла в пределах от 0 до 5» с помощью range и выводит на экран за две строчки:

```
for i in range(5):
    print(i)
```

На экране будут выведены цифры от 0 до 5:

```
0
1
2
3
4
```

Разберемся подробнее что же такое функция range(), выведите на экран результат range(5):

```
print(range(5))
```

Интерпретатор выведет:

```
range(0, 5)
```

У range есть несколько параметров, такие же как и у среза – range(START, STOP, STEP):

- START – откуда перебор начинается, необязательный параметр, по умолчанию 0;
- STOP – до куда перебор будет выполняться, обязательный параметр;
- STEP – шаг перебора.

Например, сделаем вывод всех чисел, кратных тройке в пределах двадцати всего в две строчки:

```
for i in range(3, 20, 3):  
    print(i)
```

Программа верно выведет значения:

```
3  
6  
9  
12  
15  
18
```

Требуется отметить, что функция range не создает список, в Python для диапазона range заведен отдельный тип:

```
print(type(range(5)))
```

Интерпретатор сообщит:

```
<class 'range'>
```

Во второй версии Python действительно range возвращал список чисел, который перебирался, но при большом диапазоне, это становилось накладно, программа занимала много памяти.

Диапазон можно преобразовать в список, используя функцию list:

```
range_list = list(range(5))  
print(range_list)
```

На экран будет выведен список от 0 до 5:

```
[0, 1, 2, 3, 4]
```

Т.е. с помощью range можно не только изменять параметр цикла определенным образом, но и получать списки чисел.

Попрактикуемся:

Задание 1. Посчитайте факториал чисел от 1 до 10 с помощью for и range.

Задание 2. Пробегитесь по списку с помощью цикла for и range. Выведите на экран слова, в которых содержатся комбинации букв «ан», «по» и «ие».

Ввод пользователя	Результат работы программы
бившим, двоеточие, мареограф, заваливанием, понимающий, провёртываемой, удлинялся, понимание	Слова с искомой комбинацией: 1. двоеточие; 2. заваливанием; 3. понимающий; 4. понимание.

9.2. Перебор элементов с их индексом - enumerate

Представим ситуацию, что при переборе элементов вам нужно знать на каком индексе находится элемент и сообщать об этом? Конечно, можно просто создать переменную вне цикла и менять её в цикле:

```
products = ['apples', 'fish', 'meat']

index = 0
for product in products:
    print(f'{index}: {product}')
    index += 1
```

Но для этого можно использовать функцию для итерируемых объектов enumerate(), которая возвращает сразу и индекс, и сам элемент списка, поэтому они будут идти друг за другом через запятую:

```
products = ['apples', 'fish', 'meat']

for index, product in enumerate(products):
    print(f'{index}: {product}')
```

```
for <индекс эл-та>, <эл-т> in enumerate(<последовательность>):  
    ...
```

```
table_header = [  
    'Наименование направления подготовки, специальности',  
    'Форма обучения',  
    '"№ п/п в т.2"',  
    'Наименование профиля, специализации, магистерской программы',  
    'ФГОС 3++',  
    'ОПОП на сайт (описание)',  
    'Учебный план',  
    'Календарный учебный график',  
    'Аннотации',  
    'РП',  
    'ФОС',  
    'Аннотации практики',  
    'РП практики',  
    'ФОС практики',  
    'Методические (ГИА )',  
    'Рабочая пр.воспитания+календ план',  
    'Ссылка'  
]  
  
for index, head in enumerate(table_header):  
    work_cell = self.sheet.cell(row=1, column=index + 1)  
    work_cell.value = head  
    work_cell.alignment = openpyxl.styles.Alignment(wrap_text=True)
```

```
for i in range(1, 18):  
    self.sheet.cell(row=1, column=i).border = thin_border
```

9.3. Перебор двух списков в одном цикле - zip

```
def change_weights(delta, inputs, weights, alpha=0.01):  
    new_weights = []  
    for input, weight in zip(inputs, weights):  
        new_weight = weight - (delta * input * alpha)  
        new_weights.append(new_weight)  
    return new_weights
```


10. Функции

Подпрограмма — именованная, логически законченная группа операторов языка, которую можно вызвать для выполнения любое количество раз из различных мест программы. Подпрограммы оформляются в виде замкнутых участков программы, имеющих четко обозначенные вход и выход. Во множестве языков программирования работы подпрограмм реализована в виде функций.

Функция — это именованная последовательность описаний и операторов, выполняющая некоторое законченное действие. Чтобы создать функцию, нужно определить ее имя, задаваемые параметры, последовательность инструкций, которые она должна выполнять и также обозначить что должна возвращать функция. Вызывать функцию можно по её имени.

Напишем первую функцию и на её примере разберем как с ними работать:

```
def sum_of_two(x1, x2):  
    x3 = x1 + x2  
    return x3  
  
x5 = sum_of_two(2, 3)  
print(x5)
```

Первая строка в функции называется *заголовком (header)*, весь остальной её код называется *телом (body)*.

Разберем синтаксис построения и работы с функциями:

- `def` – ключевое слово, сообщающее интерпретатору, что будет объявлена функция (*function definition*);
- `sum_of_two` – имя функции, переводится как «сумма двух»;
- `(x1, x2)` – параметры функции, с помощью параметров можно передавать в функцию переменные (передаются именно адреса переменных, а не копируются их значения);

- `return` – ключевое слово, обозначает какое значение возвращает функция;
- `sum_of_two(2,3)` – вызов функции с конкретными аргументами 2 и 3.

Несмотря на то, что одной из отличительной особенностей Python является динамическая типизация, т.е. переменная связывается с типом в момент присваивания значения, а не в момент объявления переменной, мы всё равно можем указывать типы данных у них, т.е. можно явно сказать, что у переменной целочисленное значение:

```
x:int = 10  
print(x+5)
```

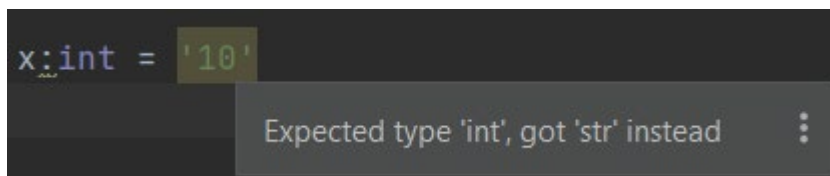
Для интерпретатора нет никакой разницы прописываете ли вы типы у переменных или нет, такой код всё равно можно будет запустить и получить ошибку:

```
x:int = '10'  
print(x+5)
```

Интерпретатор выведет следующую ошибку:

```
TypeError: can only concatenate str (not "int") to str
```

Но если вы работаете в IDE, которая может отслеживать ошибки и подсказывать как их исправить, например PyCharm, тогда вы заметите выделение '10', при наведении будет подсказка, что вы сами, как программист, задумали, что у `x` будет тип данных `int`, а не `str`:



Возвращаясь к функциям, можно задавать типы данных как у параметров, так и сообщать тип возвращаемого значения функции с помощью стрелочки `->`.

Создадим функцию, которая принимает только целые значения в качестве двух аргументов, а возвращает строку:

```
def typed_function(arg1:int, arg2: int) -> str:
    answer = f'{arg1} + {arg2} = {arg1+arg2}'
    return answer

ans = typed_function(3,2)
print(ans)
```

Результат работы программы:

```
3 + 2 = 5
```

При этом, работая в IDE при попытке передать переменную типа, отличного от того, что определен для аргументов функции, вам будет выведена подсказка о том, что неверный тип передаваемого значения. Также и при попытке использовать возвращаемого значения не по назначению его типа.

Еще один пример написания функций, в которой возвращается одновременно два значения, тогда в переменной s основной программы будет записан список из двух значений.

```
def first_func(x,y):
    d = x + y
    print(f'Hello {d}')
    return d, x

s = first_func(3,2)
print(s)
```

Результатом работы программы будет:

```
Hello 5
(5, 3)
```

Указать строки документирования docstring.

```
def sum_of_two(x1: int, x2: int) -> int:
    '''
        Наша первая функция, позволяет
        складывать два значения
    '''
    x3 = x1 + x2
    return x3

x5 = sum_of_two(2,3)
print(x5)
```

```
print(help(sum_of_two))
```

Функции также могут ничего не возвращать, тогда при описании в заголовке мы сообщаем, что функции возвращает тип None. Такие функции имеют больше информативный характер, например, для вывода сообщения пользователю:

```
def check_e(word:str) -> None:
    if 'e' in word:
        print('В слове есть e!')
```

Попрактикуемся:

ПРИМЕЧАНИЕ:

В каждом из заданий ниже также необходимо документировать каждую из функций – описывать типы входных и выходных значений в заголовке, а также не забыть о строках документирования.

Задание 1. Напишите функцию (sum_list), которая позволяет вычислить сумму всех элементов списка и примените её для подаваемых на вход списков, пока не будет встречен ноль.

Ввод пользователя	Результат работы программы
22, 36, -1, 56, 11 77, 52, -1, 77, 99, 13, -5 -5, -1, -11, 90, 120, 0, 61 0	Сумма списка 1: 124 Сумма списка 2: 312 Сумма списка 3: 254

Задание 2. На вход подается строка слов, записанных через запятую. Напишите функции, которые решают следующие задачи:

- check_lenght - проверка есть ли в слове минимум 4 буквы;

- `is_palindrome` - проверка является ли слово палиндромом;
- `get_iwords` - определить слова, которые если прочитать обратно, являются словами, которые также присутствуют в списке;
- `get_tripled_letters` - определить в каких словах идут подряд три одинаковые буквы, при этом указав повторяю букву.

Ввод пользователя	Результаты работы программы
Введите последовательность из слов: вишня, лес, шалаш, змееед, орнамент, гвоздика, сел, глаз, оса, солнце, мед, плита	Разбираемое слово – вишня – содержит больше трех букв – не является палиндромом – если обратить, не является словом в списке – три подряд одинаковые буквы не идут Разбираемое слово – лес – содержит меньше четырех букв – не является палиндромом – если обратить, является словом в списке «сел» – три подряд одинаковые буквы не идут Разбираемое слово – шалаш – содержит больше трех букв – является палиндромом – если обратить, является словом в списке «шалаш» – три подряд одинаковые буквы не идут Разбираемое слово – змееед – содержит больше трех букв – не является палиндромом – если обратить, не является словом в списке – идут три подряд одинаковые буквы «е» ...

Задание 3. Требуется реализовать функции транслитерации и сокращения наименований направлений, передавая последовательно в которые наименование направления и профиль, возможно получить следующее сокращенное наименование направления.

Функция транслитерации (*to_translit*) получает на вход строку (наименование направления + профиль), а на выходе возвращает другую строку, уже на латинице, а также исключая знаки препинания.

Функция сокращения наименования (*to_short*) получает на вход строку (не обязательно на латинице) и выполняет сокращение, оставляя только цифры, первые буквы слов (каждая из них будет с большой, кроме слова «и»), а также заменяя пробелы на нижние подчеркивания.

Входные значения	Выходные значения
Введите наименование направления: 25.05.03 Техническая эксплуатация транспортного радиооборудования Профиль направления: «Радиосвязь и электрорадионавигация морского флота»	Сокращенное наименование направления: 250503_TETR_RiEMF

Задание 4. Робот-красильщик раскрашивает поле NxN в случайном порядке он выбирает куда ему двигаться и закрасить эту клетку или нет.

Робот должен реализовывать функции:

- `create_field` – создать поле;
- `random_move` – сделать случайное движение (по столбцу или по строке);
- `paint_cell` – закрасить клетку;
- `show_field` – отобразить поле.

Создаваемое поле можно представить в виде двумерного списка (матрицы), например, 10x10, в котором незакрашенные клетки представляются нулём, а закрашенные единицей.

Обозначим переменную поля именем `field`, тогда все поле можно представить так:

[illegible]

Но что если задача изменится и нужно будет уже поле 100x100? Поэтому и нужно создавать поле с помощью функции «create_field», которая сама создает поле размерностью NxN, принимая в качестве аргумента значение.

10.1. Обязательные и необязательные параметры.

При вызове функции пользователем некоторые аргументы обязательно задавать, в то время как другие – нет. В последнем случае это означает, что у функции задано значение параметра *по умолчанию*.

Например, доработаем функцию `sum_of_two`, добавив третий параметр, который будет иметь значение по умолчанию:

```
def sum_of_two(x1: int, x2: int, x3: int=10) -> int:
    '''
    Позволяет складывать три значения
    '''
    x3 = x1 + x2 + x3
    return x3

# Вызов функции со значением по умолчанию
x5 = sum_of_two(2,3)
```

```
print(x5)

# Вызов функции с новым значением у x3
X6 = sum_of_two(4, 5, 6)
print(x6)
```

Задание 1. Требуется доработать задание 2 из «Раздела 10: Функции» - вышеуказанные функции должны возвращать булевы (bool) значения (True или False), тогда их можно будет повторно использовать не только для вывода сообщений на экран.

Опцию отображения сообщений можно реализовать с помощью добавления параметра *show_messages* со значением по умолчанию False. Например, это можно сделать так для функции *check_length*:

```
def check_lenght(word: str, show_messages=False) -> bool:
    '''
    Проверка длины
    '''
    if len(word)>3:
        if show_messages:
            print('Содержит больше 3 букв')
        return True
    else:
        if show_messages:
            print('Содержит меньше 4 букв')
        return False
```

Изменив таким образом функции, вы сможете с помощью них, например, выполнить фильтр слов по каждой функции.

10.2. Функции, как независимые сущности

(НЕ ЗАБУДЕМ ОБ ЭТОМ, Я ПОДГОТОВЛЮ МАТЕРИАЛ)

Интересно разобрать функции, как независимые сущности

10.3. Рекурсивные функции

Лучше не использовать, но посмотреть что такое можно

11. Проект: Шифрование и дешифровка сообщений

Криптография — это наука о безопасном общении посредством использования кодов и шифров. Код заменяет целые слова другими словами; шифр беспорядочно перемешивает или заменяет буквы в словах (поэтому технически азбука Морзе, или морзянка, — это в действительности шифр Морзе).

Одной из целей криптографии является использование ключа для шифровки читаемого открытого текста в нечитаемый шифротекст, а затем расшифровки его обратно в открытый текст. Цель криптоанализа — расшифровать шифры и коды, не зная их ключа или алгоритма шифрования.

11.1. Маршрутный шифр

Маршрутный перестановочный шифр (route transposition cipher) – один из способов записи сообщения с помощью перестановочного шифров. Идея заключается в том, чтобы записать текст по определенному маршруту, подобно движению змеи или зигзага.

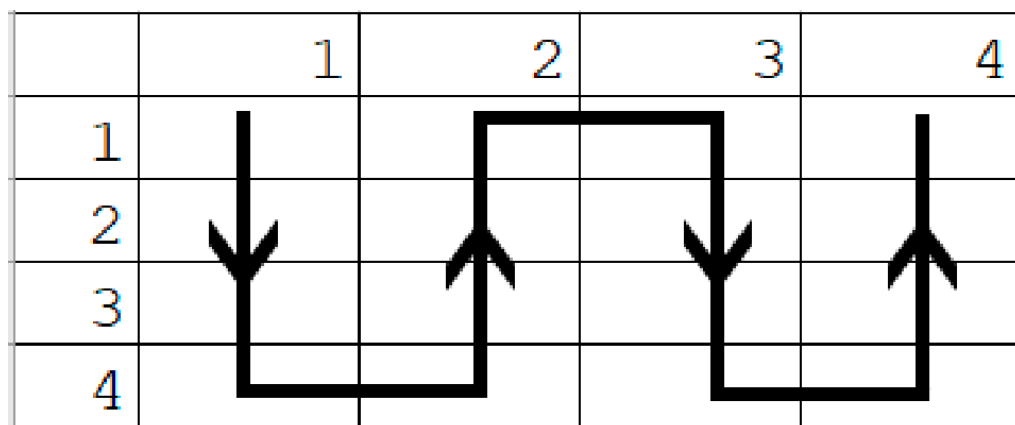
Разъяснение работы алгоритма

Возьмем текст «Маршрутный шифр», уберем пробелы и переведем в один регистр (либо верхний, либо нижний), тогда сообщение станет «МАРШРУТНЫЙШИФР».

Типом шифрования будет змеиный вертикальный маршрут сверху-слева.

Затем нужно определить ширину таблицы (по вертикали из-за выбранного выше типа), например, 4.

Маршрут следования шифрования таков:



По вышеуказанному маршруту расставим буквы сообщения:

	1	2	3	4
1	М	Н	Ы	–
2	А	Т	Й	–
3	Р	У	Ш	Р
4	Ш	Р	И	Ф

Теперь представим зашифрованное сообщение, прочитав его сверху-вниз и слева-направо: «МНЫІАТЙРУШРШРИФ».

Пояснение: При заданном ключе 3 количество строк было бы 3, а количество столбцов.

11.2. Шифр жердевой изгороди

Шифр жердевой изгороди (rail fence) – один из классических перестановочных шифров, такое имя он заслужил из-за сходства алгоритма шифрования с изгородью, у которой перегородки располагаются горизонтально зигзагом.



Разъяснение работы алгоритма

Возьмем для примера текст «ИЗГОРОДЬ», зашифрованное сообщение будет записано зигзагом по пути с N уровнями, например, ключ будет равен 3:

	1	2	3	4	5	6	7	8
1	И	–	–	–	Р	–	–	–
2	–	З	–	О	–	О	–	Ь
3	–	–	Г	–	–	–	Д	–

Затем сообщение читается по строкам, итоговым зашифрованным сообщением является «ИРЗООБГД».

Обратный ход работы алгоритма

Под «обратным ходом» понимается способ построения зигзага снизу слева, а не сверху слева. Пример построения таблицы шифрования для обратного хода ($N=3$):

	1	2	3	4	5	6	7	8
1	–	–	Г	–	–	–	Д	–
2	–	З	–	О	–	О	–	Ь
3	И	–	–	–	Р	–	–	–

Зашифрованное сообщение будет «ГДЗООЬИР»

11.3. Шифрование перестановочной матрицей

Под шифрованием перестановочной матрицей понимают перестановку по столбцам, или же прямоугольную перестановку, идея которой заключается в записи сообщения в таблицу/прямоугольник, а затем в перестановке столбцов таблицы в соответствии с ключом.

Разъяснение работы алгоритма

Для начала зададим ключ шифрования, например «СЛОВО» и составим таблицу:

	Перед сортировкой по алфавиту	После сортировки по алфавиту
Ключевое слово	СЛОВО	ВЛООС
Порядок столбцов	1 2 3 4 5	4 2 3 5 1

Затем составим таблицу, в которую запишем сообщение для шифровки «ПЕРЕСТАНОВКА», при этом количество столбцов для записи равно длине ключа:

1	2	3	4	5
П	Е	Р	Е	С
Т	А	Н	О	В
К	А	–	–	–

Затем переставим столбцы в соответствии с сортировкой ключа:

4	2	3	5	1
Е	Е	Р	С	П
О	А	Н	В	Т
–	А	–	–	К

Таким образом, зашифрованным сообщением будет «ЕЕРСПОАНВТАК».

Обратный ход работы алгоритма

Под обратным ходом имеется ввиду процесс написания сообщения в прямоугольник, начиная снизу справа:

1	2	3	4	5
–	–	–	А	К
В	О	Н	А	Т
С	Е	Р	Е	П

Тогда, при использовании того же ключа и его перестановок, как было в примере выше:

4	2	3	5	1
А	–	–	К	–
А	О	Н	Т	В
Е	Е	Р	П	С

Зашифрованным сообщением получится «АКАОНТВЕЕРПС».

11.4. Шифр redefence

Шифр redefence является усложненной версией шифра жердевой изгороди, здесь порядок шифрования определяется ключевым словом.

Разъяснение работы алгоритма

Для примера также возьмем сообщением для шифрования «ИЗГОРОДЬ», а ключевым словом «СОН».

Составим таблицу, в которой отсортируем буквы ключевого слова в алфавитном порядке, что определит порядок строк у шифра:

	Перед сортировкой по алфавиту	После сортировки по алфавиту
Ключевое слово	СОН	НОС
Порядок столбцов	1 2 3	3 2 1

Далее распишем сообщение таким же способом, как и у жердевой изгороди:

	1	2	3	4	5	6	7	8
1	И	–	–	–	Р	–	–	–
2	–	З	–	О	–	О	–	Ь
3	–	–	Г	–	–	–	Д	–

После этого нужно отсортировать столбцы сообщения в порядке, определенным ключевым словом – 3, 2, 1:

	1	2	3	4	5	6	7	8
3	–	–	Г	–	–	–	Д	–
2	–	З	–	О	–	О	–	Ь
1	И	–	–	–	Р	–	–	–

Таким образом, зашифрованным сообщением будет «ГДЗООЬИР».

11.5. Варианты для выполнения самостоятельной работы

11.5.1. Варианты средней сложности

ПРИМЕЧАНИЕ:

В ходе выполнения вариантов средней сложности требуется реализовать функцию *to_encrypt*, которая принимает строку и ключ на вход и выполняет её шифрование, возвращая новую строку.

Сообщение для шифрования/дешифровки может состоять из символов русского алфавита, пробелов и нижнего подчеркивания.

Перед шифрованием сообщение предварительно обрабатывается – все буквы переводятся в верхний регистр, а также исключаются все пробелы.

В зависимости от варианта, ключ может состоять либо из числа, либо из некоторого слова, содержащего символы русского алфавита.

Также добавьте в файл функцию *info*, которая будет возвращать пару – ФИО студента и его вариант, например:

```
def info():  
    return ['Маркова Анна Александровна', 17]
```

Формат входных/выходных данных

Работа программы с пользователем на примере жердевой изгороди (прямой ход):

Ввод данных	Вывод на экран
Введите сообщение : Изгородь Введите ключ : 3	Результат работы шифрования методом жердевой изгороди (прямой ход) : ИРЗООЬГД

Вариант 1. Напишите программу, которая зашифрует сообщение с помощью горизонтального зигзаг-маршрута снизу-слева и ключа (передаваемое число).

Ниже представлена схема маршрута шифрования сообщения с ключом 4 (кол-во строчек, столбцы вычисляются адаптивно):

	1	2	3	4
1				
2				
3				
4				

Пример таблицы шифрования сообщения «Маршрутный шифр» с помощью данного способа:

	1	2	3	4
1	Ф	Р	–	–
2	Ы	Й	Ш	И
3	Р	У	Т	Н
4	М	А	Р	Ш

Тогда итоговым сообщением будет «ФРЫЙШИРУТНМАРШ».

Вариант 2. Напишите программу, которая зашифрует сообщение с помощью змеино-вертикального маршрута снизу-справа и ключа (передаваемое число).

Ниже представлена схема маршрута шифрования сообщения с ключом 4 (кол-во строчек, столбцы вычисляются адаптивно):

	1	2	3	4
1				
2				
3				
4				

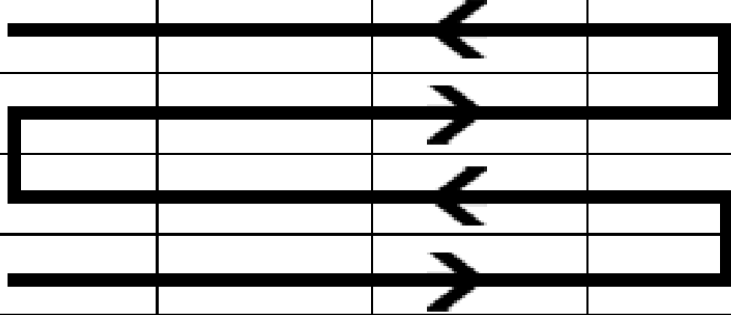
Пример таблицы шифрования сообщения «Маршрутный шифр» с помощью данного способа:

	1	2	3	4
1	Ф	И	Р	Ш
2	Р	Ш	У	Р
3	–	Й	Т	А
4	–	Ы	Н	М

Тогда итоговым сообщением будет «ИНШШТРРЙУАФЫРМ».

Вариант 3. Напишите программу, которая зашифрует сообщение с помощью змеино-горизонтального маршрута снизу-слева и ключа (передаваемое число).

Ниже представлена схема маршрута шифрования сообщения с ключом 4 (кол-во столбцов, строки вычисляются адаптивно):

	1	2	3	4
1				
2				
3				
4				

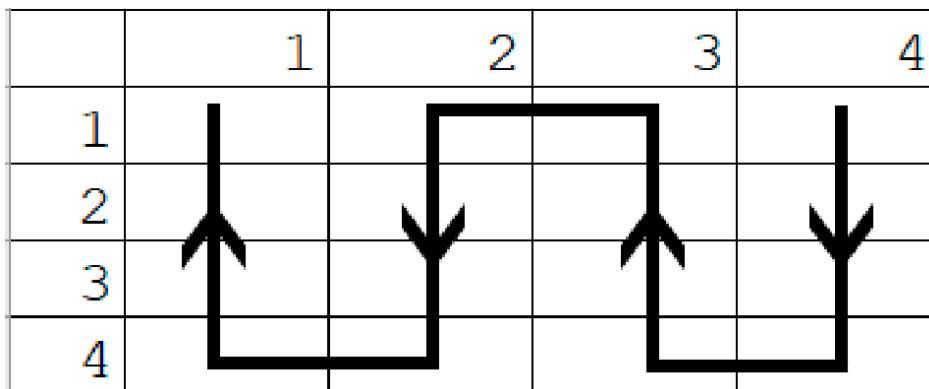
Пример таблицы шифрования сообщения «Маршрутный шифр» с помощью данного способа:

	1	2	3	4
1	–	–	Р	Ф
2	Ы	Й	Ш	И
3	Н	Т	У	Р
4	М	А	Р	Ш

Тогда итоговым сообщением будет «РФЫЙШИНТУРМАРШ».

Вариант 4. Напишите программу, которая зашифрует сообщение с помощью змеино-вертикального маршрута сверху-справа и ключа (передаваемое число).

Ниже представлена схема маршрута шифрования сообщения с ключом 4 (кол-во строчек, столбцы вычисляются адаптивно):

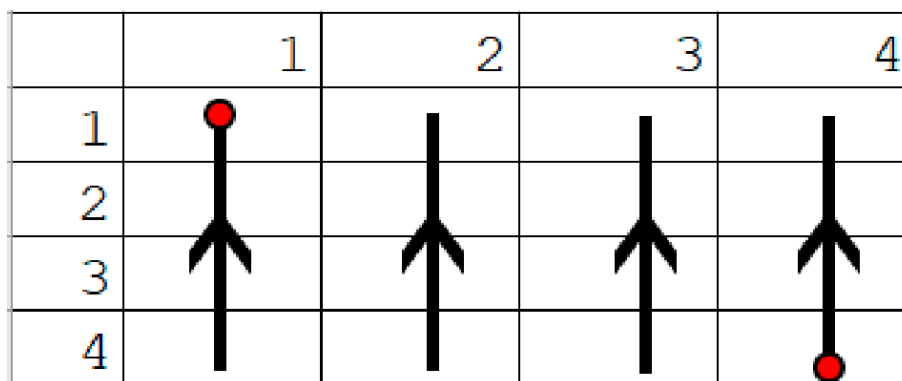


Пример таблицы шифрования сообщения «Маршрутный шифр» с помощью данного способа:

	1	2	3	4
1	–	Ы	Н	М
2	–	Й	Т	А
3	Р	Ш	У	Р
4	Ф	И	Р	Ш

Вариант 5. Напишите программу, которая зашифрует сообщение с помощью вертикального зигзаг-маршрута снизу-справа и ключа (передаваемое число).

Ниже представлена схема маршрута шифрования сообщения с ключом 4 (кол-во строчек, столбцы вычисляются адаптивно):



Пример таблицы шифрования сообщения «Маршрутный шифр» с помощью данного способа:

	1	2	3	4
1	–	И	Н	Ш
2	–	Ш	Т	Р
3	Р	Й	У	А
4	Ф	Ы	Р	М

Вариант 6. Написать программу, которая выполняет шифрование прямым ходом (слева сверху) шифра жердевой изгороди и ключа (передаваемое число).

Вариант 7. Написать программу, которая выполняет шифрование обратным ходом (слева снизу) шифра жердевой изгороди и ключа (передаваемое число).

Вариант 8. Написать программу, которая выполняет шифрование с помощью перестановочной матрицы (прямой ход) и ключа (передаваемое слово).

Вариант 9. Написать программу, которая выполняет шифрование с помощью перестановочной матрицы (обратный ход) и ключа (передаваемое слово).

Вариант 10. Написать программу, которая выполняет шифрование с помощью шифра redefence прямым ходом (изгородь строится сверху-вниз) и ключа (передаваемое слово).

Вариант 11. Написать программу, которая выполняет шифрование с помощью шифра redefence обратным ходом (изгородь строится снизу-вверх) и ключа (передаваемое слово).

11.5.2. Варианты повышенной сложности

ПРИМЕЧАНИЕ:

В ходе выполнения вариантов повышенной сложности требуется реализовать две функции:

- ***to_encrypt***, которая принимает строку и ключ на вход и выполняет её шифрование, возвращая новую строку;
- ***to_decrypt***, которая принимает зашифрованную строку и ключ, и расшифровывает сообщение, возвращая дешифрованный вариант.

Сообщение для шифрования/дешифровки может состоять из символов русского алфавита, пробелов и нижнего подчеркивания.

Перед шифрованием сообщение предварительно обрабатывается – все буквы переводятся в верхний регистр, а также исключаются все пробелы.

В зависимости от варианта, ключ может состоять либо из числа, либо из некоторого слова, содержащего символы русского алфавита.

Также добавьте в файл функцию ***info***, которая будет возвращать пару – ФИО студента и его вариант, например:

```
def info():  
    return ['Маркова Анна Александровна', 17]
```

Формат входных/выходных данных

Работа программы с пользователем на примере жердевой изгороди (прямой ход):

Ввод данных	Вывод на экран
Введите сообщение : Изгородь	Результат работы шифрования методом жердевой изгороди (прямой ход) :
Введите ключ : 3	ИРЗООЬГД
	Результат работы дешифрования : ИЗГОРОДЬ

Вариант 12. Написать программу, которая расшифровывает сообщение, которое шифруется способом из Варианта 1.

Вариант 13. Написать программу, которая расшифровывает сообщение, которое шифруется способом из Варианта 2.

Вариант 14. Написать программу, которая расшифровывает сообщение, которое шифруется способом из Варианта 3.

Вариант 15. Написать программу, которая расшифровывает сообщение, которое шифруется способом из Варианта 4.

Вариант 16. Написать программу, которая расшифровывает сообщение, которое шифруется способом из Варианта 5.

Вариант 17. Написать программу, которая расшифровывает сообщение, которое шифруется способом из Варианта 6.

Вариант 18. Написать программу, которая расшифровывает сообщение, которое шифруется способом из Варианта 7.

Вариант 19. Написать программу, которая расшифровывает сообщение, которое шифруется способом из Варианта 8.

Вариант 20. Написать программу, которая расшифровывает сообщение, которое шифруется способом из Варианта 9.

Вариант 21. Написать программу, которая расшифровывает сообщение, которое шифруется способом из Варианта 10.

Вариант 22. Написать программу, которая расшифровывает сообщение, которое шифруется способом из Варианта 11.

12. Работа с файлами

12.1. Текстовые файлы

Большинство программ, с которыми мы встречались до сих пор, временны в том смысле, что они работают в течение короткого времени и дают некоторый результат, но, когда они заканчивают выполнение, их данные исчезают.

Если вы запустите программу снова, она начнет с нуля. Другие программы персистентны (persistent): они работают в течение длительного времени (или постоянно); хранят хотя бы часть своих данных в постоянном хранилище (например, на жестком диске); и если их закрыть или перезапустить, они начнут выполнение с того места, где остановились.

Примерами постоянных программ могут служить операционные системы, которые работают все время, пока компьютер включен, и веб-серверы, которые постоянно ожидают запросов в сети.

Один из самых простых способов сохранять данные в программах — это чтение и запись текстовых файлов.

Открытие файла осуществляется с помощью функции `open`:

`open('new_file.txt', 'w')`, где:

- `'new_file.txt'` — это название и расширение файла;
- `'w'` — режим работы с файлом.

При открытии файла нужно будет объяснить интерпретатору что программа с ним собирается делать, обозначим три из них важных для нас:

- `'r'` — открытие на чтение файла;
- `'w'` - открытие на запись в файл. Если в нём что-то уже было, содержимое будет удалено;

- ‘a’ – открытие на дозапись, информация добавляется в конец файла;

Для чтения содержимого файла нужно при открытии файла задать режим чтение – «r» и применить метод текстового потока `read`:

```
f = open('new_file.txt', 'r', encoding='utf-8')
file_text = f.read()
f.close()
print(file_text)
```

Для того чтобы создания нового файла или перезаписи старого нужно открывать файл в режиме записи – «w»:

```
text_to_write = 'Текст, который будет записан в файл'
f = open('new_file.txt', 'w', encoding='utf-8')
f.write(text_to_write)
f.close()
```

Метод `close` обозначается как «заккрытие файла», что говорит об окончании его использования. Важно не забывать закрывать файл, если он больше не требуется на данном этапе выполнения программы.

Открытие файла с определенной кодировкой выполняется с помощью указания значения параметру *encoding*:

```
open('new_file.txt', 'w', encoding='utf-16')
```


Попрактикуемся:

ПРИМЕЧАНИЕ:

Для выполнения заданий (3-4) нужно будет пользоваться текстовыми файлами, которые доступны по адресу:

https://drive.google.com/drive/folders/1EGTeEylzex-ugcO2y8u-L3SuHyZ60lMV?usp=share_link

Задание 1. Написать программу, которая сохраняет 10 случайных чисел в интервале от 0 до 100, написанных через запятую с пробелом, в отдельный файл с названием «*random_10.txt*».

Задание 2. Вручную создайте файл «*products_names.txt*», который будет содержать 10 наименований продуктов. Программа должна перечислить все продукты, содержащиеся в файле, пронумеровав их.

Пример:

```
1. мясо;  
2. рыба;  
3. сыр;  
...  
10. апельсин.
```

Задание 3. Имеется файл «random_text.txt», требуется написать программу, которая выведет и сохранит в отдельный файл только email-адреса.

Email-адресом является строка, которая содержит больше одного символа перед собакой «@», а также два набора символов, разделенных точкой:

<набор символов>@<набор символов>.<набор символов>

*** Доработайте задание так, чтобы проверка строк и формирование списка адресов были вынесены в функцию. Пусть программа принимает последовательно файлы random_text_1.txt, random_text_2.txt и random_text_3.txt.**

Задание 4. Требуется написать генератор email-адресов в заданном формате. Пользователь задает число от 1 до 10 – сколько случайных адресов он хочет получить.

Формат результата работы генератора:

<название почтового ящика>@<домен 2 уровня>.<домен верхнего уровня>

Правила для формирования каждой части электронного адреса следующие:

- дан файл «domains_wiki.txt», который содержит домены верхнего уровня, нужно верно получить подстроку домена. Сохраните список всех доменов в отдельный файл и далее загружайте именно его;
- дан файл «dictionary_45_000.txt», который содержит 45 000 английских слов, случайное слово оттуда будет являться доменом 2 уровня;
- написать функцию, которая будет случайно генерировать название почтового ящика (длина от 1 до 20) – набор из английских букв и чисел.

На выводе программа должна верно строить предложение «Случайно сгенерированные N адресов».

Ввод пользователя	Результат работы программы
Сколько адресов сгенерировать : 3	Случайно сгенерированные три адреса: 34lu3y6wcvvc@performance.ky rry55df1eb2cx4@clemency.br c7fhoz71j13rvc3y@spartan.co

12.2. Ключевое слово with

with open

12.3. Работа с модулем os

Было бы отлично добавить работу с модулем os.

```
os.getcwd()
shutil.copy(
os.rename
os.listdir()
os.path.isfile()
os.remove
```

Переименование всех файлов по одному формату

Функция, которая по заданному пути выделит что является папкой, а что файлом

Функция, которая будет в глубину смотреть все папки и все файлы выносить в отдельную папку в корне. Наименование файлов должно основываться на их содержимом.

13. Словарь

Словарь (англ. «**dictionary**») — изменяемая неупорядоченная структура данных, которая позволяет хранить пары «ключ — значение».

Ключом могут быть многие хешируемые значения – на данный момент для нас это значит, что ключами могут быть *не только* целые числа, но и вещественные, а также строки. В целом, любой неизменяемый тип данных может быть ключом (например, список ключом быть не может).

Значением же может быть что угодно, в т.ч. изменяемые типы данных.

Создать пустой словарь можно двумя способами – с помощью пустых фигурных скобок или вызова функции dict:

```
new_dict = {}
new_dict = dict()
```

Ключи в словарях играют ту же роль, что и индексы, например, в списках – с помощью ключей можно обратиться к конкретным значениям словаря.

Формально структуру словаря можно описать так:

<наименование словаря> = {<ключ_1>: <значение_1>, ...
<ключ_N>: <значение_N>}

Можно создать словарь, у которого уже будут *изначально* записаны некоторые данные:

```
# Данные о продуктах и их кол-ве на складе
products_data = {
    'Апельсины': 15,
    'Сыр': 20,
    'Мёд': 5,
    # Запятую у последней пары можно здесь оставить, чтобы потом
    # было проще что-то дописать и не забыть о ней
}
```

13.1. Получение значений из словаря

Попробуем узнать сколько апельсинов имеется в словаре *products_data*, для этого обратимся к нему по ключу «Апельсины»:

```
oranges_number = products_data['Апельсины']
print(oranges_number)
```

13.2. Добавление пар в словарь

Для добавления новой пары «ключ-значение» в существующий словарь, достаточно обратиться по ключу, которого **ещё нет** в словаре. Например, добавим новые значения в *products_data*:

```
products_data['Шампиньоны'] = 30
products_data['Шпроты'] = 13
products_data['Яблоки'] = 8
print(products_data)
```

При запуске такого кода вы увидите следующее содержимое словаря:

```
{ 'Апельсины': 15, 'Сыр': 20, 'Мёд': 5, 'Шампиньоны': 30,
  'Шпроты': 13, 'Яблоки': 8 }
```

Если же обратиться по ключу, который **уже есть** в словаре, вы просто измените значение по этому ключу, например:

```
products_data['Мёд'] = 20 # Привоз мёда в магазин
products_data['Сыр'] -= 5 # Продажа сыра покупателям
print(products_data)
```

Значения словаря `products_data` будут изменены:

```
{ 'Апельсины': 15, 'Сыр': 15, 'Мёд': 20, 'Шампиньоны': 30,  
'Шпроты': 13, 'Яблоки': 8 }
```

13.3. Удаление ключа из словаря

Для того, чтобы удалить ключ из словаря, можно воспользоваться уже знакомым по спискам методом **pop**:

```
products_data.pop('Сыр')  
print(products_data)  
print(products_data['Сыр'])
```

13.4. Работа с циклом for

С помощью цикла `for` мы будем получать значения ключей:

```
for product in products_data:  
    print(product)
```

Результат работы:

```
Апельсины  
Сыр  
Мёд  
Шампиньоны  
Шпроты  
Яблоки
```

Попрактикуемся

Доступ к тестовому файлу по ссылке:

<https://drive.google.com/drive/folders/1oEE6SnAuZwy9A-vVh9om1bcERzNyhDVx?usp=sharing>

Дан файл `input.txt`, в котором содержится `html`-код одной из таблиц. Значения первого столбца содержат в себе наименования дисциплин. Требуется восстановить нумерацию дисциплин, а также отсортировать наименование дисциплин по алфавиту.

Перебор ключей можно выполнять и без метода .keys()

Распаковка ключей для сортировки по наименованию ключа

```
self.t_full_list.setRowCount(len(self.result_data_dict.keys()))
sorted_keys = [*self.result_data_dict] # Распаковка
sorted_keys.sort()
```

Объединение словарей не всегда может быть надежным средством

```
# Попытка сделать красивое объединение словарей, но ненадежное, т.к. что-то терялось
# self.Process.connected_docs = self.result_data_dict | self.left_data_dict | self.right_data_dict |

if self.data_source == 'connected_docs':
    merged_dict = {}
    for dict_part in [self.result_data_dict, self.left_data_dict, self.right_data_dict, self.no_keys_dict]:
        for key in dict_part:
            if key not in merged_dict:
                merged_dict[key] = dict_part[key].copy()
            else:
                for doc_type in dict_part[key]:
                    if merged_dict[key][doc_type] == '':
                        merged_dict[key][doc_type] = dict_part[key][doc_type]
```

14. Множества

15. Многофайловый проект

Точка входа

Разница простого написания вызова функции и `if __name__ == 'main':` в том, что при подключении файла в первом случае код просто исполнится.

Сказать про точку - файл подключается из этой же папки, где находится этот файл.

```
from django.urls import path
from django.conf import settings
from django.conf.urls.static import static
from .views import index, news_preview, acc
from form.authorization.views import author
```

16. Проект: Создание текстовой игры

В данном разделе вам предлагается применить все пройденные выше темы на практике – создайте свою первую текстовую игру. Можно выбирать тему как из предложенных, так и придумать что-то свое, но требования к разработанной программе одни – вы должны для начала описать все возможности в данной игре и её цель – составить техническое задание, а только потом писать код программы.

С помощью составления ТЗ вы должны определить рамки работы, чтобы этот проект не делался бесконечно, наша задача сделать что-то простенькое, но со вкусом.

Вариант 1. Сделать игру-приключение о кроте, который копает подземные ходы и находит различные вещицы и сокровища. Также может встречать других персонажей, которые могут просить главного героя дать им какую-либо из найденных вещей взамен за награду (валюту или что-либо другое). В игре можно прокачивать экипировку крота, чтобы более эффективно раскапывать новые ходы, а также прокапываться через более прочную руду.



Игра будет видом сверху, где будут отображаться клетки, на которых может что-либо показываться.

Возможно сохранение прогресса игрока в файл.

Вариант 2. Создать градостроительный симулятор, в котором игрок может строить различные здания для развития виртуального города. Каждый дом после постройки будет приносить прибыль, но его постройка также стоит денег и времени. Игрок может пропускать время. Построение нескольких зданий разных типов может положительно или негативно влиять на прибыль и популяцию города.



Возможно сохранение прогресса игрока в файл.

Вариант 3. Вдохновляясь книгой Сетона-Томпсона «Жизнь Серого Медведя» (Например, по этой ссылке лежит в открытом доступе <http://www.serann.ru/text/zhizn-serogo-medvedya-8761>) предлагается сделать игру, в которой игрок сможет прочувствовать на себе все радости и невзгоды медвежьей жизни.



Игра начинается со стадии медвежонка, малыш следует и учится у медведицы как добывать еду.

В игре должно быть как минимум 4 карты, которые позволяют медведю добывать пищу в любое время года:

«Ранней весной хорошо пировать в местах, где пасется скот и водятся лоси. Здесь много бывает трупов погибших зимой животных. Ранним летом лучше всего добывать пищу на пригретых солнцем склонах холмов. Там можно найти разные вкусные корни и дику репу. В конце лета поспевают много ягод на кустах в низинах вдоль рек. Осенью же в сосновых лесах есть много пищи, и там можно запастись жиром на зиму.»

Карта может являться как текстовым описанием того, что происходит рядом, так и отображением схематичного изображения местности.

Медвежонок, наблюдая за матерью, получает знания о внешнем мире. Например, под камнями могут быть насекомые, которыми он может насытиться, в реке можно поймать рыбу и др..

Также как и Уэб, главный герой произведения, медвежонок может потерять мать и теперь будет учиться различным урокам на основе своей интуиции и выборов игрока.

Прогресс игрока сохраняется в файл, пока игрока нет, медвежонок всё равно старается выживать в лесу. Несколько созданных медвежат могут

встречаться в лесу и взаимодействовать. У медвежат может быть различное поведение, которое определяет их взаимодействие с остальными животными.

Основные показатели медвежонка:

- Здоровье – определяет то насколько здоров медведь и сколько он ещё проживет. На здоровье влияют также показатели еды и воды;
- Еда – медведи должны есть, чтобы поддерживать здоровье и силы;
- Вода – медведи должны также и пить, пить можно откуда угодно, они ведь не хрупкие люди;
- Настроение – складывается из показателей здоровья, еды и воды;
- Характер – является отражением опыта взаимодействий медведя с окружающим миром.

Поведение при встречах с другими животными/людьми определяется характером и настроением медведя. Даже при самом хорошем настроении медведь со скверным характером может напасть и, наоборот, даже самое плохое настроение мирного медведя не выведет из себя.

При загрузке игры спустя некоторое продолжительное время медведь расскажет что он увидел в лесу и задумается правильно ли он поступил в той или иной ситуации, так игрок может влиять на действия медведя.

Вариант 4. Создать модель колонии муравьев.

Вариант 5. Игра про управление пекарней.

Вариант 6. Подobie игры Beholder, но в текстовом варианте. Также можно наблюдать за жильцами и выведывать их секреты.

Различные задания, которые надо разобрать:

Списки:

В некоторых следующих задачах вам придется пользоваться случайными числами. Первые несколько задач подготавливают к решению следующих.

1. Дан список, можно считать, что это список чисел. Проверить, есть ли в нем два одинаковых элемента.
2. Дан список и натуральное число n . Проверить, что в списке встречаются все числа от 1 до n .

1. **Парадокс дней рождения.** В году 365 дней ($d = 365$), в группе 30 студентов ($n = 30$). С какой вероятностью найдутся два студента с одним днем рождения?

1. Заведите функцию, которая ставит один эксперимент. На вход получает d, n . Генерирует случайные n дней рождения, проверяет, не совпали ли два, возвращает логическое значение.
2. Создайте функцию, которая получает на вход d, n и количество экспериментов e , например, $e = 1000$. Запускаем эксперимент e раз. Возвращаем вероятность совпадения дней рождения, она равна тому, сколько раз дни рождения совпали, делить на количество экспериментов.

2. **Яйца с сюрпризом.** Представим, что мы покупаем яйца с сюрпризом, в которых есть ($n = 10$) возможных игрушек внутри. И мы хотим собрать коллекцию из всех игрушек. Сколько понадобится купить яиц, в среднем, чтобы собрать полную коллекцию?

1. Делаем функцию, которая проводит единичный эксперимент. Она получает на вход число n , потом последовательно генерирует случайный номер игрушки от 1 до n и повторяет процесс, пока не сгенерируются все номера от 1 до n . Например, при $n = 3$ могут сгенерироваться числа 1, 3, 3, 3, 1, 1, 3, 2. Получив два, функция останавливает процесс генерации, потому что получены все номера от 1 до 3.

Функция должна вернуть номер шага, на котором сгенерировались все номера. В примере это 8.

2. Сделайте функцию, которая получает на вход n и количество экспериментов e , например, $e = 1000$. Запускаем эксперимент e раз и считаем среднее значение количества шагов:

$$\frac{steps_1 + steps_2 + \dots + steps_e}{e}.$$

Это среднее значение и нужно вернуть.

Ваша задача будет состоять в том, чтобы отобрать из списка всех русских слов [russian.7z](#) те слова, которые подходят под условия ниже. Кроме того, чтобы укоротить вывод, давайте считать, что в этой задаче нам интересны только слова минимум из четырех букв, и в которых кроме маленьких русских букв нет никаких других символов.

Не забудьте, что каждая задача — это отдельная функция. Она должна принимать на вход список всех русских слов для анализа, и должна возвращать список из подходящих слов, или что-то другое, указанное в условии.

1. Сделайте функцию чтения файла. Она получает на вход имя файла и возвращает список слов из файла, подходящих под основное условие: длина хотя бы 4, все символы — это маленькие буквы.

Еще сделайте функцию распечатки списка слов. Она должна принимать на вход список слов, ничего не возвращать и печатать слова на экране. В строку, в столбик, как удобно.

2. Палиндромы. Это слова, которые читаются с начала и с конца одинаково;
3. В которых хотя бы 4 гласных, и все гласные в слове одинаковые;
4. 20 самых длинных русских слов;
5. В которых есть три одинаковых буквы подряд;
6. Слова, которые, если их прочесть с конца, тоже оказываются русскими словами, т.е. тоже находятся внутри файла. Но при этом, эти слова не должны быть палиндромами.
7. Слова, в которых буквы увеличиваются. Т.е. список `sorted(слово)` должен быть такой же, как список букв слова.
8. В каком слове чаще всего повторяется буква? И какая эта буква? Т.е. посчитайте для каждого слова, какая буква в нем встречается больше всего раз, и запомните, сколько раз она встретилась. После чего отберите слова, где это значение максимально. Верните структуру такого типа: `(2, [("мама", "м"), ("голова", "о"), ...])`

9. Найдите самое длинное русское слово, из которого можно удалять буквы по одной, чтобы в результате слова оставались русскими. Например, слово мама, мам, ам, а.

Подсказка: отсортируйте слова по возрастанию длины, для каждого слова нужно сохранять, подходит ли оно под условие. Попробуйте удалить у слова по каждой букве и проверьте, не подходит ли под условие хотя бы какое-то из полученных коротких слов.

Напишите функцию `avoids()`, которая принимает слово и строку запрещенных букв и которая возвращает `True`, если в слове нет запрещенных букв.

Измените вашу программу, чтобы предложить пользователю ввести строку запрещенных букв, а затем печатает количество слов, которые не содержат ни одного из них. Можете ли вы найти сочетание из пяти запрещенных букв, которое исключает наименьшее количество слов?